

Instituto Politécnico Nacional
Escuela Superior de Cómputo



Simulación de sistemas complejos mediante autómatas celulares neuronales.

Presentan
Angeles Lopez Erick Jesse[1]

Fecha: 28 de octubre de 2025

[1]eangelesl1700@alumno.ipn.mx

Índice general

1. Introducción **7**

1.1. Resumen 7

1.2. Planteamiento del problema 7

1.2.1. Preguntas de investigación 9

1.3. Objetivo 9

1.3.1. Objetivos específicos 10

1.4. Hipótesis 10

1.5. Justificación 10

1.6. Alcances y limitaciones 11

2. Marco teórico **13**

2.1. Modelado de sistemas complejos 13

2.2. Autómatas celulares 16

2.2.1. Vecindad 17

2.2.2. Estados 18

2.2.3. Condiciones de frontera 19

2.2.4. Regla local 20

2.2.5. Clasificación 23

2.3. Redes neuronales 26
2.3.1. Inteligencia artificial y aprendizaje supervisado 26
2.3.2. Neurona biológica 28
2.3.3. Neurona artificial 29
2.3.4. Redes neuronales 31
2.3.5. Aprendizaje en redes neuronales 31
2.4. Autómatas celulares neuronales 35

3. Estado del Arte **38**
3.1. Simulación mediante autómatas celulares 39
3.2. Morfogénesis 41
3.2.1. NCA Isotrópicos 42
3.2.2. Autoclasificación 43
3.2.3. Representaciones tridimensionales 46
3.3. Mesh Neural Cellular Automata 49
3.4. Modelos generativos 50
3.5. Representaciones y aplicaciones adicionales 53

4. Marco metodológico **56**
4.1. Diseño de la investigación 56
4.1.1. Diseño experimental 56
4.2. Enfoque de la investigación 57
4.3. Población 58
4.4. Muestra y muestreo 58
4.5. Tipo de investigación 59

4.6. Técnicas e instrumentos de recolección de datos 59
4.6.1. Técnicas de recolección de datos 59
4.6.2. Instrumentos de recolección de datos 60
4.7. Procedimiento 60
4.8. Técnicas y herramientas de procesamiento de datos 61
4.8.1. Procesamiento de datos cuantitativos 61
4.9. Contextualización de la tecnología 62
4.9.1. Fundamentos y contexto de las tecnologías 62
4.9.2. Entornos de trabajo 66

5. Análisis de software **69**

5.1. Metodología de desarrollo 69
5.2. Reglas de negocio 70
5.3. Requerimientos 71
5.3.1. Requerimientos funcionales 71
5.3.2. Requerimientos no funcionales 72
5.3.3. Requerimientos de sistema 72
5.4. Análisis de riesgos 73
5.4.1. Identificación de riesgos 73
5.4.2. Probabilidad e impacto 74
5.4.3. Plan de mitigación 77
5.4.4. Plan de contingencia 78
5.5. Estudio de factibilidad 80
5.6. Casos de uso 81
5.7. Diccionario 81

5.7.1. Entradas 81
5.7.2. Mensajes 83
5.7.3. Interfaces graficas 83
5.8. Especificaciones de casos de uso 84
5.8.1. Definir vecindad 84
5.8.2. Definir condición de frontera 86
5.8.3. Definir estados 86
5.8.4. Definir función de transición 88
5.8.5. Establecer colores 89
5.8.6. Iniciar simulación 90
5.8.7. Pausar simulación 91
5.8.8. Establecer velocidad 91
5.8.9. Simular un paso 92
5.8.10. Retroceder un paso 93
5.8.11. Reiniciar simulación 94
5.8.12. Cambiar estado de una célula 95
5.8.13. Importar evolución 96
5.8.14. Exportar evolución 97

6. Diseño de software **98**
6.1. Arquitectura 98
6.1.1. Vista 98
6.1.2. Modelo 99
6.1.3. Controlador 99
6.2. Interfaces de usuarios 99

Referencias **102**

Capítulo 1

Introducción

1.1. Resumen

En el presente documento se presenta la primera etapa de una investigación orientada a la simulación de sistemas complejos en espacios bidimensionales mediante autómatas celulares neuronales. El objetivo principal es utilizar una red neuronal para definir la función de transición de un autómata celular, con el fin de modelar y simular las representaciones de sistemas complejos sobre una red regular bidimensional.

Los resultados incluyen el desarrollo del marco teórico, que recopila el conjunto de definiciones formales necesarias; el estado del arte, donde se analizan investigaciones previas relacionadas con diferentes enfoques y configuraciones; la descripción del marco metodológico, en donde se especifican las técnicas, herramientas y procedimientos utilizados; y los avances obtenidos hasta el momento, que abarcan la implementación y documentación de herramientas para la simulación de autómatas celulares en dos dimensiones.

1.2. Planteamiento del problema

La simulación de sistemas complejos ha surgido como una herramienta indispensable para la comprensión y el análisis de fenómenos en diversas áreas del conocimiento, desde la física y la biología hasta las ciencias sociales y económicas. Tradicionalmente, los autómatas celulares han servido como un modelo matemáti-

co fundamental para estudiar estos sistemas mediante la representación de procesos dinámicos.

Los autómatas celulares son un modelo matemático definido como una 4-tupla que contiene:

- Un espacio de celdas en un dominio discreto.
- Un conjunto de estados posibles.
- Una vecindad que define las dependencias entre células.
- Una función de transición que determina cómo evolucionan simultáneamente las células[**?**, **?**].

Ejemplos clásicos de autómatas celulares incluyen el Juego de la Vida de John Conway y los 256 autómatas elementales descritos por Stephen Wolfram. Estas herramientas han sido aplicadas en múltiples contextos como la simulación de incendios, modelado de distribuciones poblacionales, simulación de partículas y el análisis de ciclos climáticos [**?**]. No obstante, una limitación crítica de los modelos convencionales reside en la rigidez de sus funciones de transición; estas deben definirse de manera estática y anticipada, restringiendo la posibilidad de adaptarse a condiciones imprevistas o cambios en el entorno. Esta limitación motiva la búsqueda de nuevas metodologías capaces de incorporar mayor flexibilidad y adaptabilidad en la simulación de sistemas dinámicos.

En este contexto, se busca la integración de redes neuronales en el marco de los autómatas celulares. Las redes neuronales, inspiradas en el funcionamiento del cerebro y su capacidad de aprendizaje, ofrecen la posibilidad de redefinir las reglas de transición de forma dinámica. Esto permite que el autómata celular se ajuste de manera autónoma a partir de la experiencia y el entrenamiento [**?**].

El presente proyecto de investigación propone superar las limitaciones de los autómatas celulares mediante la integración de redes neuronales en la definición de las reglas de transición, dando origen a los *autómatas celulares neuronales*. En este enfoque, las reglas de actualización dejan de ser fijas y adquieren la capacidad de aprender y adaptarse a partir de los datos, lo que permite que el sistema se ajuste de forma dinámica ante comportamientos complejos y entornos variables [**?**, **?**]. Esta aproximación no solo busca ampliar el campo de aplicación de los autómatas celulares, sino que también busca abrir nuevas oportunidades para modelar y simular fenómenos de formas realistas.

Dadas las infinitas configuraciones de autómatas celulares, este proyecto se limita a un espacio bidimensional finito (con condiciones de frontera) y un conjunto finito y numerable de estados. Adicionalmente, se busca experimentar con diferentes vecindades incluyendo Moore y Newman. Aunque el objetivo no es determinar las condiciones para la evolución de las células, si se busca realizar pruebas bajo diferentes configuraciones que permitan mostrar los resultados sin comprometer su calidad.

Por otro lado, la red neuronal recibe como parámetros los estados de las celdas vecinas y devuelve el nuevo estado de la celda a la cual se aplique la función de transición. Este proceso es el mismo para todas las células.

Es por esto que surge la pregunta central de esta investigación: ¿Cómo los autómatas celulares neuronales pueden simular sistemas complejos en un espacio bidimensional? A partir de este cuestionamiento, surgen las siguientes preguntas de investigación.

1.2.1. Preguntas de investigación

- ¿Qué es un sistema complejo?
- ¿Qué es simular?
- ¿Como se simula un sistema complejo?
- ¿Qué es un autómata celular?
- ¿Que es una red neuronal?
- ¿Qué es un autómata celular neuronal?
- ¿Como simular un sistema complejo?

1.3. Objetivo

Desarrollar un sistema de simulación basado en autómatas celulares neuronales bidimensionales que permita analizar la generación, mantenimiento, recuperación y propagación de patrones de información en entornos complejos y dinámicos.

1.3.1. Objetivos específicos

- Diseñar e implementar un simulador de espacios discretos bidimensionales que sirva de plataforma para estudiar los autómatas celulares neuronales.
- Entrenar y evaluar el desempeño de los autómatas celulares neuronales bajo diferentes configuraciones.
- Replicar configuraciones clásicas con comportamientos y soluciones conocidas.
- Analizar y comparar diferentes comportamientos con base en la red neuronal establecida.

1.4. Hipótesis

Si un sistema de simulación está basado en autómatas celulares neuronales bidimensionales, entonces es posible analizar y reproducir la generación, mantenimiento y propagación de patrones de información en entornos complejos y dinámicos.

1.5. Justificación

Los autómatas celulares son modelos lo bastante simples para un análisis teórico, pero lo suficientemente complejos para exhibir fenómenos emergentes. Definir las reglas de comportamientos erráticos para ser sincronizados resulta en una tarea compleja [**?**].

Por este motivo, el presente proyecto de investigación surge ante la necesidad de superar las limitaciones inherentes a los modelos tradicionales de autómatas celulares, los cuales se caracterizan por depender de reglas de transición fijas y predeterminadas. Este enfoque restrictivo, si bien ha sido útil para modelar ciertos fenómenos, no siempre captura la complejidad y la adaptabilidad de sistemas dinámicos presentes en entornos naturales, sociales y tecnológicos.

Cada autómata celular resuelve un problema, sin embargo, su definición matemática implica que existe una infinitud de problemas a resolver. Por lo que realizar configuraciones hasta obtener el resultado deseado resulta un proceso computacionalmente inviable [**?**].

En este sentido, se propone la integración de redes neuronales en la definición de las reglas de transición, con el objetivo de desarrollar un modelo de *autómata celular neuronal* capaz de aprender y adaptarse de manera autónoma a cambios en el entorno. Esto no solo amplía el alcance y la aplicabilidad de los autómatas celulares, sino que también abre un novedoso campo de estudio en el que se combinan herramientas de inteligencia artificial y simulación de sistemas complejos. La principal innovación de este enfoque reside en reemplazar las funciones de transición estáticas por modelos ajustables. Mediante técnicas de aprendizaje automático, la función de transición puede entrenarse con datos representativos, permitiendo que el sistema se adapte de manera autónoma a los cambios en el entorno [**?**].

Adicionalmente, este proyecto profundiza en técnicas de simulación, modelado matemático y redes neuronales, integrando conocimientos de ciencias de la computación, vida artificial y matemáticas.

La experiencia adquirida durante el desarrollo e implementación de estos modelos abrirá puertas a futuras investigaciones y proyectos en el campo de la inteligencia artificial. Además, la creación de un entorno de código abierto contribuirá al avance del estado del arte y facilitará que otros investigadores puedan extender y aplicar estos conceptos.

En síntesis, este proyecto busca contribuir al avance de la simulación de sistemas complejos mediante la creación de un modelo flexible, adaptable y extensible de autómatas celulares neuronales, consolidando un punto de encuentro entre la teoría de sistemas dinámicos y las técnicas modernas de aprendizaje automático.

1.6. Alcances y limitaciones

Alcances

- El estudio se limita a la simulación en espacios bidimensionales finitos con condiciones de frontera establecidas en una red regular definida.
- Se emplean redes neuronales supervisadas para la definición de la función de transición.
- Se analizarán comportamientos de patrones conocidos y emergentes bajo distintas configuraciones de estados y vecindades.

Limitaciones

- No se contempla la generalización del modelo a diferentes dimensionalidades o topologías de red.
- El análisis se centra en la simulación de sistemas complejos, sin abordar la optimización computacional del entrenamiento.
- Los resultados se interpretan únicamente dentro del contexto de los parámetros y condiciones experimentales definidos.

Capítulo 2

Marco teórico

2.1. Modelado de sistemas complejos

Las colonias de hormigas, el mercado bursátil, el clima, el sistema nervioso, las células y los organismos vivos son algunos ejemplos de sistemas complejos (CS, *Complex Systems*). Estos se caracterizan por tener un conjunto de elementos que interactúan entre si con el fin de alcanzar un objetivo [**?**, **?**].

Los CS tienen dos implicaciones fundamentales: (1) Existe una influencia mutua entre sus elementos, donde el cambio afecta inevitablemente al resto de elementos, y (2) persiguen un objetivo [**?**].

Definición 1. *Un sistema complejo es una entidad compuesta de partes interconectadas, de tal forma que el comportamiento colectivo es mas que la suma de sus componentes individuales.*

Para entender el comportamiento de un sistema es necesario conocer los elementos y sus interacciones. Es decir, se requiere conocer los estados, valores instantáneos de todos los elementos; y sus transiciones, cambios dinámicos en los estados.

Los CS se caracterizan principalmente por:

- **No linealidad**: No cumple el principio de superposición, es decir, el resultado no puede expresarse como la suma de las respuestas individuales a cada estimulo por separado. Resulta difícil predecir su evolución dinámica futura [**?**, **?**].

- **Retroalimentación**: Las salidas pasadas influyen en las siguientes salidas. Este comportamiento hace alusión a la memoria del sistema [**?**].
- **Orden espontaneo**: Existe un orden emergente resultado de las interacciones locales no coordinadas el cual no puede ser descrito mediante sus interacciones locales [**?**].

  Aparición (*emergence*): Entidades, patrones, comportamientos y propiedades emergen de la interacción de entidades mas pequeñas, las cuales no exhiben estos atributos [**?**].

  Elementos múltiples: Compuesto de múltiples elementos relativamente idénticos con interacciones locales [**?**].

Resulta evidente que predecir el comportamiento de estos sistemas es una tarea complicada. Una aproximación es el uso de modelos científicos y matemáticos, los cuales se construyen para desarrollar procesos de inferencia sobre aspectos observados en la vida real. El modelo de inferencia se realiza para entender el sistema real observado.

El proceso, de manera simplificada, se compone de tres fases, como se observa en la figura 2.1:

- Se abstrae el sistema real a un modelo, el cual puede trascender a otros sistemas reales. Por ejemplo, para calcular la caída libre se reduce la complejidad ignorando factores como el color o la forma.
- El modelo realiza un proceso de inferencia que busca averiguar las implicaciones derivadas de las premisas.
- Los resultados se utilizan para analizar, interpretar y aplicar a un sistema real. Esta aplicación no requiere ser practica, es una relación entre las inferencias obtenidas con el modelo y el sistema real.

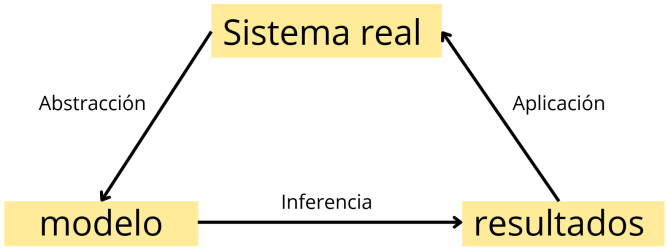


Figura 2.1: Representación grafica del modelado científico. Este proceso busca relacionar eventos del mundo real utilizando un modelo que realiza una inferencia.

Sin embargo, estos modelos suelen suelen carecer de las herramientas necesarias que les permiten deducir las implicaciones lógicas necesarias. Es aquí donde la computación permite agilizar este proceso realizando simulaciones computacionales.

Las computadoras permite explorar y analizar modelos formales para resolver modelos matemáticos. Ademas, dado que un modelo computacional puede ser definido mediante un modelo matemático, el rol de la computación es simplificar este proceso.

Los resultados que se obtienen al ejecutar la simulación computacional se derivan de aplicar reglas algorítmicas, por lo que se requiere expandir el proceso de abstracción y diseñar un modelo formal que sera ejecutado por la computadora, como se muestra en la figura 2.2.

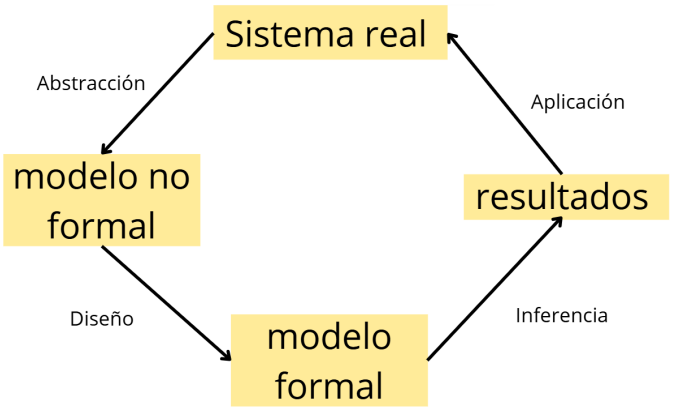


Figura 2.2: Representación grafica del modelado computacional. Relaciona eventos del mundo real utilizando un modelo computacional que ayuda a agilizar el proceso de inferencia.

Se requiere de un experto en el sistema real que pueda abstraer el problema y un modelador que formalice dicha abstracción para ejecutarla en una computadora que infiera los resultados, mismos que utilizara nuevamente el experto en el sistema real para entablar la relación con el sistema real [**?**].

Para modelar sistemas complejos existen múltiples herramientas, una de ellas son los autómatas celulares [**?**], los cuales siguen la caracterización de los sistemas complejos: elementos idénticos con interacciones locales que pueden generan orden espontaneo, comportamientos emergentes e incluso retroalimentados.

2.2. Autómatas celulares

Un autómata celular (CA, *Cellular Automata*), concebido por John von Neumann [1], es una colección de células que pueden adquirir un estado y que se agrupan en una red. El CA evoluciona en pasos de tiempo, donde cada célula cambia su estado en función de los estados de su vecindad. Durante la evolución, el CA cambia de una configuración a otra [2, 3, **?**]. Tradicionalmente, se considera una red regular de células, un conjunto finito de estados y evoluciones en pasos discretos de tiempo.

Definición 2. *Un autómata celular es una 4-tupla* $(\mathcal{L}, \mathcal{S}, \mathcal{N}, \mathcal{R})$ *donde:*

- $\mathcal{L} \subseteq \mathbb{Z}^D$ *es el espacio celular D-dimensional. Un conjunto de células se colocan en el espacio* $\mathcal{L}$ *para formar una red.*
- $\mathcal{S}$ *es un conjunto de estados. Cada célula solo puede adquirir un valor de un conjunto finito de diferentes valores:*

$$\sigma_{\vec{i}}(t) \in \mathcal{S} \equiv \{0, 1, 2, \ldots, k-1\},\ \forall \vec{i} \in \mathcal{L}$$

  Donde $\vec{i} \in \mathcal{L}$ *es una célula en el espacio celular,* $\sigma_{\vec{i}}(t)$ *es el estado de la i-ésima célula en el tiempo t y* $\mathcal{S}$ *es un anillo conmutativo finito*[1]*, generalmente definido por los enteros modulo k (*$\mathbb{Z}_k$*).*
- $\mathcal{N} = (\vec{v}_1, \vec{v}_2, \ldots, \vec{v}_m)$ *es el conjunto de vectores de desplazamiento que definen la vecindad de una célula en* $\mathcal{L}$*. Asi, los vecinos de las células ubicadas en* $\vec{i} \in \mathcal{L}$ *se encuentran en las posiciones* $\vec{i} + \vec{v}_j, \forall j \in \{1, 2, \ldots, m\}$
- $\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$ *es la regla local del autómata:*

$$\sigma_{\vec{i}}(t+1) = \mathcal{R}(\{\sigma_{\vec{i}+\vec{v}_j}\}_{j=1}^{m})$$

  donde $\mathcal{R}$ *es la función de transición local que asigna el nuevo estado de una célula a partir de los estados de sus m vecinos.*

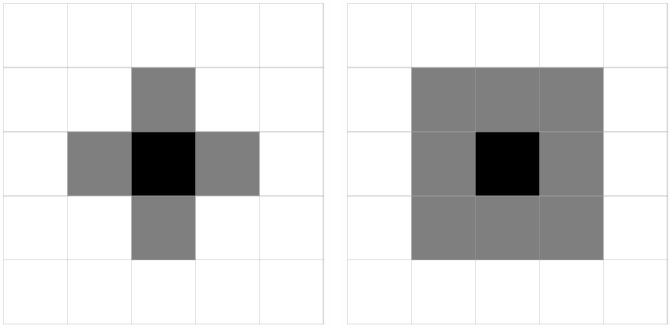
Esta formulación sigue de manera general a [**?**, **?**, **?**].

[1] Un anillo conmutativo finito es un conjunto con suma y multiplicación, donde la suma se comporta como una suma normal (conmutativa e inversa), la multiplicación también es conmutativa y ambas operaciones cumplen la propiedad distributiva.

2.2.1. Vecindad

La vecindad de una célula está estrechamente relacionada con la dimensionalidad del CA. John von Neumann propone una vecindad para $\mathbb{Z}^2$ de cinco células formada por ella misma y las cuatro células ortogonales más cercanas [**?**] como se observa en la figura 2.3a. Esta vecindad es expandida por Moore añadiendo los cuatro vecinos diagonales más cercanos, dando una vecindad de nueve elementos [**?**]. Esta vecindad, representada por la figura 2.3b, fue utilizada por John Conway en el "Juego de la Vida" (GoL, *Game of Life*) [**?**].

La figura 2.3c representa una vecindad de radio uno en un espacio unidimensional, es decir, todas aquellas células a una distancia menor o igual a uno, incluyéndose a su misma. Fue utilizada por Stephen Wolfram para definir los Autómatas Celulares Elementales (ECA) [**?**].



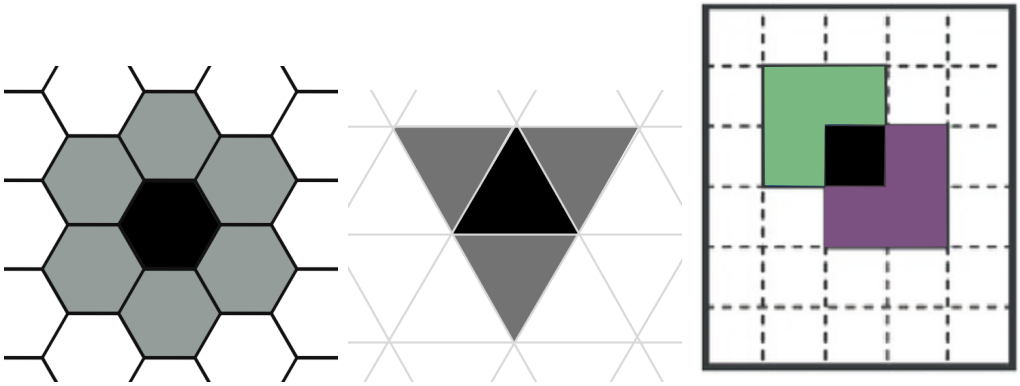
(a) Vecindad de Neumann (b) Vecindad de Moore



(c) Vecindad radio uno

Figura 2.3: Las vecindades 2.3a y 2.3b son para CA bidimensionales, la vecindad 2.3c es para CA unidimensionales. La célula negra representa la célula actual y, junto con las grises, conforma su vecindad.

Las vecindades no se limitan a una cuadrícula regular; también pueden adoptar otras formas, como las vecindades hexagonales o triangulares ilustradas en las figuras 2.4a y 2.4b. Ademas, existen vecindades con comportamiento dinámico, como la vecindad de Margolus mostrada en la figura 2.4c, cuya configuración alterna entre dos disposiciones en cada paso evolutivo [**?**].



(a) Vecindad hexagonal (b) Vecindad triangular (c) Vecindad de Margolus

Figura 2.4: Vecindades para CA bidimensionales. En 2.4a y 2.4b, la célula negra es la célula actual y las grises conforman su vecindad. En 2.4c, la célula negra es la célula actual, mientras que las células verdes y moradas representan las vecindades activas en tiempos pares e impares, respectivamente.

2.2.2. Estados

Cada célula adquiere un único estado s en cada paso de tiempo, donde s pertenece a un conjunto de estados $s \in \mathcal{S}$. El CA de John von Neumann contempla 29 estados, aunque algunos autores han propuesto variantes reducidas a cuatro estados, conservando su capacidad de autoreproducirse [**?**].

Por otro lado, GoL se compone de dos estados que denominó "vivas" y "muertas" [**?**]. De igual forma Stephen Wolfram utiliza dos estados en ECA [**?**].

Idealmente, el espacio celular $\mathcal{L}$ es infinito, lo que permitiría infinitas combinaciones de estados. Sin embargo, esto es inviable computacionalmente, por lo que, al igual que el CA de John von Neumann, se requiere establecer una restricción de finitud como lo marca [**?**].

Definición 3. *Sea $\mathcal{R}$ la regla local de un CA. Un estado $\sigma_q \in \mathcal{S}$ es inactivo si $\mathcal{R}(\sigma_q, \sigma_q, \ldots, \sigma_q) = \sigma_q$.*

Es decir, si una célula y todos sus vecinos están en estado inactivo, la célula permanece inactiva tras aplicar $\mathcal{R}$. Cabe notar que todos los vecinos deben de estar en dicho estado, por lo que, si alguno cambia, podría reactivarse.

Para trabajar computacionalmente, [**?**] define las configuraciones finitas de la siguiente manera:

t	$\vec{\sigma}(t)$										
0				$\ldots$	q	p	q	$\ldots$			
1			$\ldots$	q	p	p	p	q	$\ldots$		
2		$\ldots$	q	p	p	p	p	p	q	$\ldots$	
3	$\ldots$	q	p	p	p	p	p	p	p	q	$\ldots$

Tabla 2.1: Evolución temporal de un CA unidmensional con una configuración inicial finita. Donde p representa estados activos y q estados inactivos. Los conjuntos $\vec{\sigma}(t)$ y $\vec{\sigma}(t+1)$ son finitos.

Definición 4. *Sea $\sigma_q \in \mathcal{S}$ un estado inactivo, $\sigma_p \neq \sigma_q \in \mathcal{S}$ un estado no inactivo y sea $\mathcal{L}$ un espacio celular infinito. Se considera que un CA tiene configuración finita si solo un número finito de celdas están en estados activos σ_p.*

Si la configuración inicial es finita, entonces, la evolución en el siguiente paso de tiempo, también sera finita (ver tabla 2.1).

Sea $\mathcal{C} = \mathcal{S}^{\mathcal{L}}$ el conjunto de todas las configuraciones. Definimos $\mathcal{C}_F \subset \mathcal{C}$ como el subconjunto de todas las configuraciones finitas. Esta restricción es necesaria para hacer computacionalmente viable la simulación de la evolución de un CA.

Finalmente, resulta necesario definir un estado estable el cual servirá mas adelante para establecer una clasificación [**?**].

Definición 5. *Dado un CA y un estado $s \in \mathcal{S}$, se dice que s es un estado estable si $\mathcal{R}(s, s, \ldots, s) = s$.*

Esto es, un estado estable s que evoluciona siempre en si mismo, por lo que $\sigma(t) = \sigma(t+1)$.

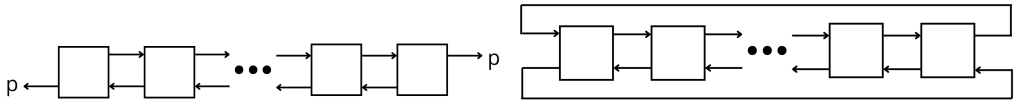
2.2.3. Condiciones de frontera

Idealmente, los CA pueden evolucionar en espacios infinitos; sin embargo, para su simulación computacional es necesario establecer límites. Estos limites, llamados condiciones de frontera, restringen el espacio celular $\mathcal{L}$ [**?**, **?**].

Definición 6. *Un CA es finito si el espacio celular $\mathcal{L} \subseteq \mathbb{Z}^D$ es finito; en caso contrario, se denomina infinito.*

En un CA finito se consideran principalmente dos tipos de condiciones de frontera: abierta y periódica [**?**].

- **Condición de frontera abierta**: Se les asigna un estado fijo a todos aquellos vecinos faltantes en las células de la frontera.
- **Condición de frontera periódica**: Los vecinos faltantes de las células en la frontera son otras células en la frontera. La figura 2.5b muestra cómo se conectan las células de la frontera en un espacio unidimensional.



(a) Condición de frontera abierta (b) Condición de frontera periódica

Figura 2.5: Condiciones de frontera de un CA unidimensional con dos vecinos. La figura 2.5a asigna un estado no inactivo al vecino faltante. La figura 2.5b entabla relación de vecindad entre las células en la frontera formando un anillo.

Las condiciones de frontera periódica relacionan a los CA finitos y los CA infinitos con configuraciones periódicas [**?**].

Definición 7. *Una configuración $\vec{\sigma} \in \mathcal{C}$ de un CA D-dimensional se dice periódica si es invariante ante un conjunto de traslaciones linealmente independientes en D direcciones. Es decir:*

$$\vec{\sigma} = T_i^{n_i}(\vec{\sigma}),\ \forall i \in \{1, 2, \ldots, D\}$$

donde $T_i^{n_i}$ denota un desplazamiento de n_i posiciones en la dimensión i.

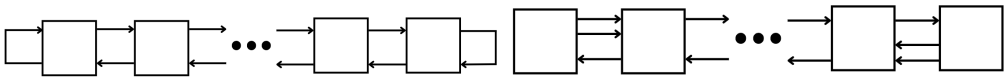
Definimos entonces a $\mathcal{C}_P \subset \mathcal{C}$ como el conjunto de todas las configuraciones periódicas de un CA. Estas configuraciones preservan la condición de frontera periódica en cada traslación [**?**].

La frontera reflectiva asigna a los vecinos faltantes el mismo estado de la célula en la frontera (figura 2.6a). En este caso, las células de la frontera son vecinas de sí mismas. La frontera reflexiva, en cambio, imita el estado de los vecinos mas próximos de la célula en el borde (figura 2.6b), generando una doble relación de vecindad en los extremos del CA [**?**, **?**].

2.2.4. Regla local

Sea $\mathcal{S}$ el conjunto de estados y $m = |\mathcal{N}|$ el numero de vecinos. La función de transición o regla local se define como

$$\mathcal{R} : \mathcal{S}^m \to \mathcal{S}$$



(a) Condición de frontera reflectiva (b) Condición de frontera reflexiva

Figura 2.6: Condiciones de frontera de un CA unidimensional con dos vecinos. La figura 2.6a muestra que las células en la frontera son vecinas de si misma, mientras que la figura 2.6b muestra que las células vecinas de las células de la frontera pueden ser múltiples vecinos.

donde $\mathcal{R}$ asigna un nuevo estado a cada célula en función de los estados de sus m vecinos (incluyéndose a sí misma si aplica).

Para CA unidimensionales, dado un rango arbitrario r, la regla de un CA unidimensional puede ser escrita como:

$$\sigma_i(t+1) = \mathcal{R}(\sigma_{i-r}(t), \ldots, \sigma_i(t), \ldots, \sigma_{i+r}(t)),\ \mathcal{R} : S^{2r+1} \to S$$

Donde el rango r especifica el tamaño de la vecindad para cada célula y cada una de ellas puede adquirir un valor k. Por esto, $\mathcal{R}$ puede ser definido especificando el valor asignado para cada una de las k^{2r+1} posibles $(2r+1)$-tuplas dada una vecindad.

Un ejemplo de regla local es GoL [**?**]. En este caso, el conjunto de estados es binario $\mathcal{S} = \{0, 1\}$, donde 0 representa células muertas y 1 representa células vivas. La regla de transición se define como:

$$\mathcal{R} : \{0, 1\}^9 \to \{0, 1\}$$

ya que la vecindad de Moore en 2D involucra 8 vecinos mas la propia célula, es decir, $m = 9$. GoL sigue las siguientes tres reglas:

- **Nacimiento**: Una célula muerta en el tiempo t nace en el tiempo $t+1$ si tiene exactamente tres vecinos vivos en el tiempo t.
- **Muerte**: Una célula viva en el tiempo t muere en $t+1$ por:
  - sobrepoblación; si hay más de tres vecinos vivos en t.
  - soledad: si hay menos de tres vecinos vivos en t.
- **Supervivencia**: Una célula viva en el tiempo t se mantiene viva en $t+1$ si tiene dos o tres vecinos vivos en t.

111	110	101	100	011	010	001	000
↓	↓	↓	↓	↓	↓	↓	↓
$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$	$\{0,1\}$

Tabla 2.2: Cada posible combinación de tres células puede adquirir uno de dos valores. Cuando la célula evaluado encaja en este patrón su valor cambia al establecido. Este conjunto de 256 forman parte del ECA.

Se puede usar la notación *Birth/survival* la cual es un dupla de listas que almacenan el numero de vecinos vivos que tiene que haber en la vecindad para nacer y para sobrevivir respectivamente. La configuración para el juego de la vida "tradicional" es $B3/S23$, es decir, se requieren 3 células vivas para nacer y se requieren 2 o 3 células vivas para sobrevivir [**?**].

Una regla es semi-totalistica si la transición depende de la suma de los valores en la vecindad, pero distingue entre diferentes posiciones, como el centro y los bordes. GoL es un ejemplo de regla semi-totalistica porque cuenta el numero de vecinos vivos, pero trata de forma distinta el estado de la célula central [**?**].

En cambio, una regla es totalistica si la transición depende unicamente de la suma de los estados en todos los vecinos, sin distinguir posiciones [**?**]. Por ejemplo, para un CA unidimensional con $|\mathcal{S}| = 3$ estados y radio $r = 1$, solo hay $(|\mathcal{S}| - 1)|\mathcal{N}| + 1 = (2-1) \cdot 3 + 1 = 4$ posibles valores de suma.

Por otro lado, una regla local arbitraria si distingue las posiciones de cada elemento en la vecindad. La configuración dada para ECA considera $k = 2$ (es decir, $\sigma_i \in \{0,1\}$), un radio $r = 1$ ($|\mathcal{N}| = 3$) y la regla $\mathcal{R} : \{0,1\}^3 \to \{0,1\}$ que se define como:

$$\phi(\sigma_{i-1}(t), \sigma_i(t), \sigma_{i+1}(t))$$

Esta dinámica esta dada por las posibles configuraciones que las tres células adyacentes pueden adquirir en el tiempo t. Podemos encontrar $k^{|\mathcal{N}|} = 2^3 = 8$ patrones posibles en la vecindad, numero correspondiente a la combinatoria de dos estados en tres posiciones. Cada que encontremos un patrón podemos asignarle un nuevo estado a $\sigma_i(t+1)$ ($\{0,1\}$), por lo que el numero total de reglas asciende a $k^{k^{|\mathcal{N}|}} = 2^8 = 256$, como se muestra en la tabla 2.2.

Esta reglas se pueden representar de forma tabular y también como un numero decimal. La tabla 2.3 muestra la asociación entre la forma tabular y la regla decimal.

Nótese que la regla decimal solo aplica cuando las condiciones son las mismas,

Binario	111	110	101	100	011	010	001	000	
Decimal	7	6	5	4	3	2	1	0	**regla**
Valor	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	
	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	0	1	1
					⋮				⋮
	0	0	0	1	1	1	1	0	30
					⋮				⋮
	0	1	1	0	1	1	1	0	110
					⋮				⋮
	1	1	1	1	1	1	1	1	255

Tabla 2.3: Los valores binarios representan las posibles configuraciones de estados que pueden aparecer en el espacio, cada uno asociado a un valor decimal y el valor es la potencia asociada a esa combinación. Para convertir la regla tabular en la decimal solo es necesario convertir el numero binario.

es decir, cuando el numero de vecinos $|\mathcal{N}|$ y estados $|\mathcal{S}|$ son invariantes, a cada regla se le puede asociar un numero entero único, pero dos reglas que difieren en vecindad o estados pueden compartir el numero decimal. Por ejemplo, considerando la regla 10 con $|\mathcal{N}| = 2$ y $|\mathcal{S}| = 2$ es 1010, la cual es completamente diferente cuando $|\mathcal{N}| = 3$ y $|S| = 2$, lo que da como resultado 000000101.

2.2.5. Clasificación

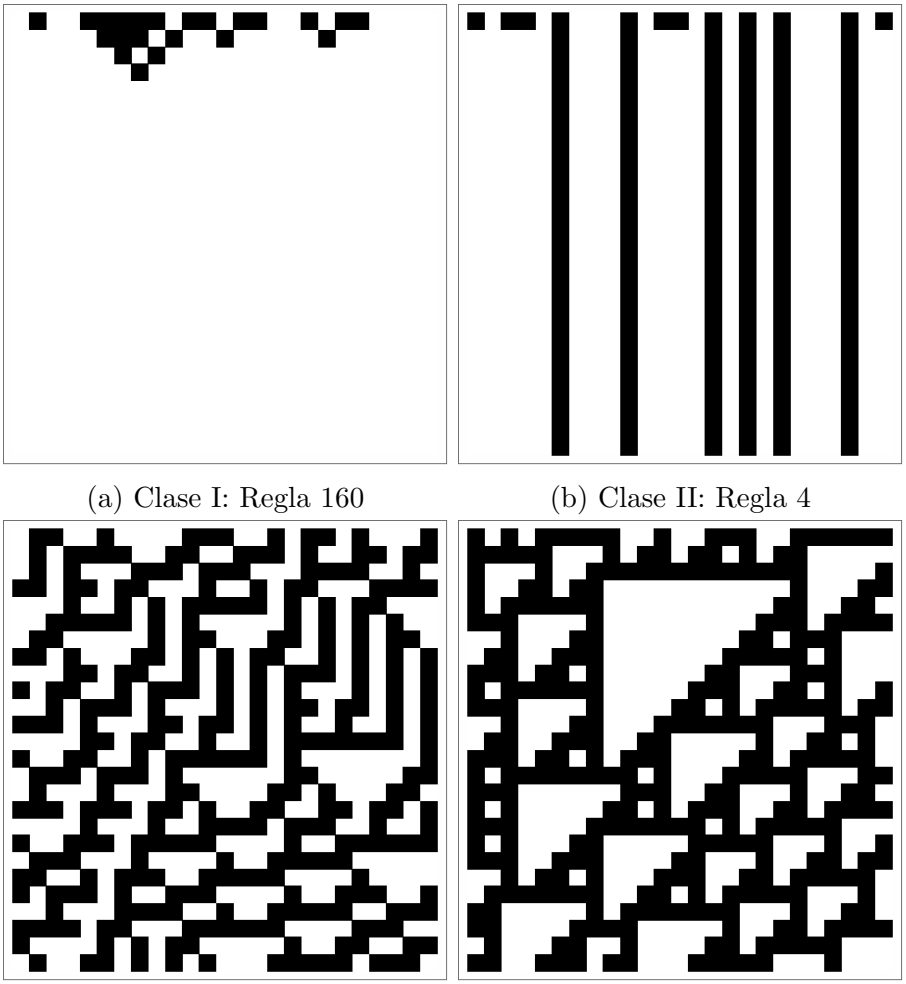
Stephen Wolfram, al estudiar ECA, descubre que existen comportamientos compartidos por ciertas reglas y que dicho comportamiento es inherente de la configuración inicial del espacio celular. Es por esto que define las siguientes clases de comportamiento, ilustradas en la figura 2.7:

I CA que evolucionan a un estado uniforme.

II CA que evolucionan a estados cíclicos aislados.

III CA que evolucionan en comportamientos caóticos.

IV CA que evolucionan en estados complejos aislados.



(a) Clase I: Regla 160

(b) Clase II: Regla 4

(c) Clase III: Regla 30

(d) Clase IV: Regla 110

Figura 2.7: Clasificación de Wolfram. las figuras evolucionan 25 pasos una configuración inicial aleatoria con condición de frontera cíclica.

Sin embargo, aunque esta clasificación funciona para ECA existen reglas bajo diferentes configuraciones que pueden pertenecer a múltiples clases o, en el peor de los casos, no pertenecen a ninguna de ellas [**?**].

Es por esto que en [**?**] proponen una nueva clasificación siguiendo ciertas limitaciones, entre las cuales están:

- Esta clasificación es para CA finitos.
- Cada clase es un subconjunto de la clase siguiente.

Definición 8. *Un CA es de clase I si existe un estado estable $s \in \mathcal{S}$ tal que todas las configuraciones finitas evolucionan a una configuración homogénea de s.*

Es decir, dada cualquier configuración inicial en un CA finito, todas las células adquieren el estado s tras cierto numero de pasos evolutivos. Esto se aplica para todas las finitas configuraciones iniciales.

Definición 9. *Un CA es de clase II si existe un estado estable $s \in \mathcal{S}$ tal que se repite a n pasos evolutivos.*

Esto implica que la evolución se repite cada n numero de pasos ($\sigma(t) = \sigma(t+n)$). Resulta evidente que la clase I esta contenida en la case II haciendo que $n = 1$, lo que implica un periodo de uno.

Una clasificación es s-finita si solo difiere del estado estable en una región finita.

Definición 10. *Un CA es de clase III si existe un estado estable $s \in \mathcal{S}$ tal que para cualquier par de configuraciones s-finitas C_1 y C_2, es decidible si C_1 evoluciona a C_2.*

Es decir, si podemos determinar en tiempo finito si el CA evoluciona de C_1 a C_2, entonces pertenece a clase III. En caso contrario, pertenece a clase IV, la cual contiene a todos los CA [**?**].

Otra clasificación considera extensiones o modificaciones en la definición que alteran el comportamiento del CA. Entre ellas se encuentran los autómatas celulares sobre grafos [4], con funciones de transición estocásticas [5], no homogéneos [6], o definidos en configuraciones con estados, espacio y tiempo continuos [7]. Particularmente relevante para esta investigación son los autómatas celulares neuronales, que emplean una red neuronal como función de transición. Antes de definirlos, es necesario comprender qué son las redes neuronales.

2.3. Redes neuronales

Las redes neuronales artificiales (ANN, *Artificial Neural Network*) es un modelo de inteligencia artificial inspirado en el pensamiento humano, mas específicamente, en como un conjunto de neuronas interconectadas comparten información.

Las ANN son una conjunto de neuronas artificiales interconectadas que realizan cálculos y transmiten información para realizar una operación, imitando el comportamiento del cerebro.

Pero para entender el modelo de ANN es necesario tener los fundamentos de inteligencia artificial y la clasificación de aprendizaje supervisado.

2.3.1. Inteligencia artificial y aprendizaje supervisado

La inteligencia artificial (AI, *Artificial Intelligence*) fue definida por primera vez por John McCarthy en 1955 como *la ciencia e ingeniería de hacer maquinas inteligentes* [**?**]. Siguiendo a [**?**, **?**, **?**], se propone la siguiente definición:

Definición 11. *La Inteligencia Artificial es la ciencia que utiliza modelos computacionales para crea sistemas capaces de realizar tareas que, si fueran hechas por humanos, requerirían inteligencia.*

Una de las ramas de la AI es el aprendizaje automático (ML, *Machine Learning*), el cual se centra en desarrollar sistemas con la capacidad de aprender en función de los datos [**?**]. Esto permite reconocer patrones y realizar predicciones a partir de grandes cantidades de datos [**?**].

Por lo que, con base en [**?**, **?**] podemos definir al aprendizaje ML como:

Definición 12. *El aprendizaje automático es la disciplina que permite a las computadoras mejorar su desempeño en una tarea mediante la experiencia datos, sin necesidad de instrucciones explícitas.*

ML funciona trabajando a través del entrenamiento de algoritmos en conjuntos de datos para conseguir un resultado esperado. Es el proceso de optimizar el modelo para que pueda predecir la respuesta correcta. A mayor calidad en los datos de entrenamiento, mayor precisión se tendrá en el modelo [**?**].

El entrenamiento se refiere al proceso de enseñarle a un modelo de inteligencia artificial a realizar una o varias tareas mediante la exposición a grandes cantidades

de datos [**?**]. Este proceso facilita a los humanos el proceso de análisis de grandes volúmenes de datos, especialmente en la era actual donde se generan más datos que nunca en la historia [**?**].

Dentro del aprendizaje automático se definen tres métodos en como una computadora puede aprender. Estos métodos son aprendizaje supervisado, no supervisado y por refuerzo.

- El aprendizaje supervisado se caracteriza por el uso de conjuntos de datos etiquetados para entrenar algoritmos que clasifiquen datos o predigan resultados con precisión. Durante el proceso de entrenamiento, el modelo recibe entradas junto con las salidas correctas, lo que le permite al modelo ajustar sus parámetros hasta que sea capaz de realizar predicciones precisas sobre nuevos datos [**?**].
- El aprendizaje no supervisado, a diferencia del aprendizaje supervisado, no necesita de un conjunto de datos etiquetados generados por el ser humano. Este tipo de aprendizaje puede descubrir patrones y estadísticas sin ninguna orientación o instrucción explícita [**?**].
- El aprendizaje por refuerzo busca imitar la forma de aprendizaje humana de prueba y error. Este tipo de aprendizaje permite a las máquinas aprender a tomar decisiones y mejorar sus resultados con base en los errores cometidos [**?**].

IBM [**?**] menciona algunos los algoritmos más comunes de aprendizaje automático:

1. **Redes neuronales:** Simulan el funcionamiento del cerebro humano, con un gran número de nodos de procesamiento conectados entre sí, llamados neuronas.
2. **Regresión lineal:** Predice valores numéricos, con base en una relación lineal entre diferentes valores.
3. **Regresión logística:** Realiza predicciones para variables de respuestas categóricas.
4. **Agrupación en clústeres:** Identifican patrones en los datos y los agrupan en centroides.

5. **Árboles de decisión:** Utilizan una secuencia ramificada de decisiones vinculadas que se representan con un diagrama de árbol.

6. **Bosques aleatorios:** Combina los resultados de varios árboles de decisión y predice un valor o clasificación.

Las redes neuronales presentan ventajas que pueden llegar a agilizar y optimizar recursos de las organizaciones que lo implementan [**?**]: mejora la productividad, reduce el error humano, mejora la disponibilidad de servicios, reducir tareas repetitivas, descubrir patrones ocultos a simple vista, entre otros.

Para comprender como las redes neuronales logran estas capacidades, es necesario explorar primero los fundamentos que inspiraron el desarrollo. En particular, su diseño es una abstracción de las neuronas biológicas, cuya estructura y funcionamiento ofrecen una analogía al procesamiento de información.

2.3.2. Neurona biológica

Las neuronas son células especializadas del sistema nervioso cuyo objetivo es recibir, procesar y transmitir información por medio de señales químicas y eléctricas [**?**, **?**].

Cada neurona se compone (principalmente) de tres elementos (como se muestra en la figura 2.8):

- Las dendritas, que reciben señales provenientes de otras neuronas.
- El núcleo o cuerpo, que integra las señales proveniente de las dendritas y produce una nueva señal eléctrica.
- El axón, el cual conduce la señal producida por el cuerpo hacia otras neuronas haciendo sinapsis, es decir, transmite la señal de una célula a otra.

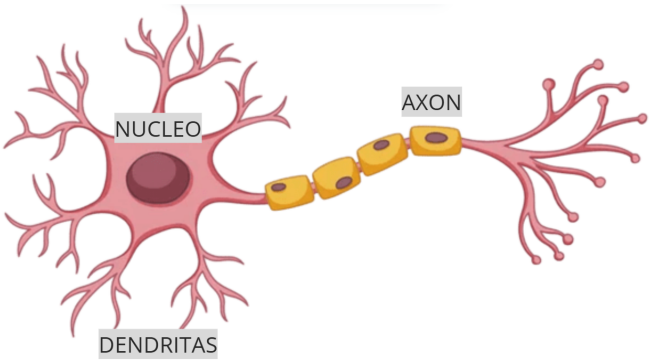


Figura 2.8: Representación de una neurona [**?**]. Se encuentra señaladas las dendritas, el núcleo y el axón, encargado de recibir, procesar y transmitir la información respectivamente.

Las redes neuronales naturales son un sistema de neuronas que permite el procesamiento cerebral de la información, son altamente complejas, no lineales y con procesamiento en paralelo.

2.3.3. Neurona artificial

La neurona artificial se considera como una unidad de procesamiento de información [**?**]. Sigue la misma estructura del modelo biológico con algunas consideraciones adicionales como lo marcan [**?**, **?**, **?**]:

- Las dendritas equivalen al conjunto de datos de entrada. Sin embargo, no toda la información tiene la misma relevancia, para esto se le asigna a cada entrada un peso sinaptico.
- El cuerpo de se sustituye por dos funciones: (1) Una función de agregación que combina todos los datos de entrada (generalmente es una combinación lineal entre las entradas y los pesos sinapticos) y (2) una función de activación que determina si la neurona esta activa y en que grado.

La cual se suele representar como en la figura 2.9.

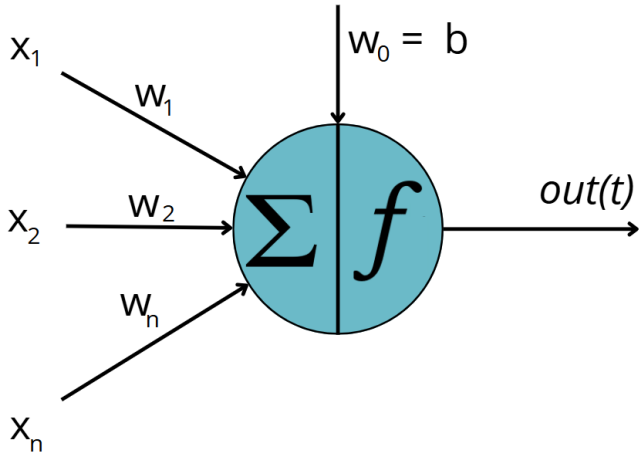


Figura 2.9: Representación grafica de una neurona artificial. Contiene las entradas $(x_1, x_2, \ldots, x_n)$, los pesos sinápticos $(w_1, w_2, \ldots, w_n)$, el sesgo (w_0 o b), función de agregación (Σ), función de activación (f) y la salida.

Definición 13. *Una neurona artificial es una función* $f : \mathbb{R}^n \to \mathbb{R}$ *dada por:*

$$f(x) = \phi(\omega(X, W, b))$$

donde:

- $X = \{x_1, x_2, \ldots, x_n\} \in \mathbb{R}$ *es el conjunto de datos que recibe cada neurona.*
- $W = \{w_1, w_2, \ldots, w_n\} \in \mathbb{R}$ *es el conjunto de pesos sinapticos que indican la importancia de cada entrada.*
- b *o* $w_0 \in \mathbb{R}$ *es un sesgo que desplaza la función de activación y controla el umbral de aceptación.*
- $\omega(X, W, b)$ *es una función de agregación* $\omega : \mathbb{R}^n \to \mathbb{R}$ *que combina las entradas. Usualmente se define como una combinación lineal:*

$$\omega(X, W, b) = \sum_{i=1}^{n} w_i x_i + b$$

- $\phi(\omega(X, W, b))$ *es la función de activación* $\phi : \mathbb{R} \to \mathbb{R}$ *que transforma* $\omega(X, W, b)$ *en una salida activada.*

La función de activación es la que determina el valor de salida de cada neurona. Este valor, normalmente, se asigna entre un umbral para que limiten el comportamiento de la red neuronal [**?**].

2.3.4. Redes neuronales

Una neurona unicamente puede realizar una operación. Para aumentar la capacidad de entender patrones mas complejos es necesario añadir múltiples neuronas interconectadas, lo que se conoce como red neuronal artificial (ANN, *Artificial Neural Network*) [**?**]. Una ANN puede definirse como un grafo dirigido [**?**]:

Definición 14. *Una red neuronal artificial es un grafo dirigido $G = (V, E)$ donde cada nodo $v \in V$ representa una neurona y cada arista $(u, v) \in E$ transporta la salida y_u desde u hasta v ponderada por el peso w_{uv}.*

Existe un conjunto de nodos de entrada $X \subseteq V$ que reciben los datos iniciales y un conjunto de nodos de salida $T \subseteq V$ cuya activación final representa la salida de la red.

La red en su conjunto implementa una función parametrizada

$$F_\theta : \mathbb{R}^n \to \mathbb{R}^m$$

donde n es el numero de datos de entrada, m el numero de datos de salida y θ el conjunto de parámetros entrenables.

Por lo que podemos concluir con la definición de [**?**], la cual dice que

Definición 15. *Una red neuronal es una función $F_\theta : \mathbb{R}^n \to \mathbb{R}^m$ donde $\theta = \{w_{uv}, b_v : (u, v) \in E, v \in V\}$ es el conjunto de parámetros entrenables.*

2.3.5. Aprendizaje en redes neuronales

Dada una ANN con los parametros entrenables totalmente configurados, es capaz de realizar su clasificación o predicción de manera correcta. Sin embargo, resulta imposible configurar estos pesos manualmente, por lo que requiere de un "entrenamiento" que ayude a definir los pesos sinápticos. A continuación se define la función de perdida, optimización y entrenamiento, elementos necesarios para obtener esa configuración especifica.

2.3.5.1. Función de perdida

La función de perdida se usa pera medir el rendimiento del modelo. Calcula la desviación que existe entre el valor predicho y el valor esperado. Esta función

puede ser un escalar o un vector que permite aproximar la tasa de error para llegar a una solución optima [**?**].

Definición 16. *Sea F_θ una ANN y $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ un conjunto de datos de entrenamiento donde $x^{(i)}$ es la entrada y $y^{(i)}$ es la salida esperada, una función de perdida es un mapeo*

$$\mathcal{L} : \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}_{\geq 0}$$

tal que mide la discrepancia entre la predicción del modelo y la salida real:

$$\mathcal{L}(F_\theta(x^{(i)}, y^{(i)}))$$

Es importante que la función de perdida sea diferenciable. El gradiente de la función de perdida permite conocer la dirección hacia la cual debe moverse para disminuir la perdida.

Las funciones de perdida se pueden utilizar para regresión y para clasificación. La primera mide el error en valores continuos, usado para estimar valores contables como el precio, tamaño, tiempo. La segunda mide el errores en valores discretos, como la clasificación o la categoría.

Diferentes funciones de perdida priorizan diferentes tipos de error, la elección de la función de perdida dependerá de la tarea que se desea realizar. Ademas, la función de perdida unicamente se limita a una muestra, para una población de muestras, se utiliza la función de costo, la cual se define a continuación [**?**, **?**].

Definición 17. *Sea F_θ una ANN y $\mathcal{L}$ un conjunto de datos de entrenamiento, la función de costo es el promedio de las perdidas:*

$$J(\theta) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(F_\theta(x^{(i)}), y^{(i)})$$

2.3.5.2. Optimización

El entrenamiento de una red consiste en ajustar los parámetros entrenables θ para minimizar la función de perdida. La optimización resulta en resolver el problema de minimización dado por:

$$\theta^* = \arg\min_\theta J(\theta)$$

Esto se puede conseguir mediante diferentes técnicas, siendo el descenso del gradiente la mas común. Este método actualiza los parámetros en la dirección

opuesta al gradiente de la función de perdida:

$$\theta^{t+1} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}(\theta^{(t)})$$

donde $\eta > 0$ es la tasa de aprendizaje. Una tasa de aprendizaje alta puede divergir en la solución, mientras que una tasa baja puede tener una convergencia lenta [**?**].

2.3.5.3. Entrenamiento

Finalmente, el entrenamiento consiste en ajustar todos los parámetros $\theta = \{W^{(l)}, b^{(l)}\}$ para minimizar la función de costos $J(\theta)$ en un conjunto de datos de entrenamiento. Este proceso se compone de las siguientes etapas:

1. Se inicializan los parámetros, generalmente de forma aleatoria: $\theta_0 \sim \mathcal{D}_{init}$
2. Se hace la propagación hacia adelante o *forward pass*, es decir, para cada entrada $x^{(i)}$ se evaluá $\hat{y}^{(i)} = F_\theta(x^{(i)})$
3. Se calcula la pérdida: $\mathcal{L}^{(i)} = \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$
4. Se calculan los gradientes propagando la información hacia atrás o *backpropagation* utilizando la regla de la cadena:

$$\nabla_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \mathcal{L}(F_\theta(x^{(i)}, y^{(i)}))$$

   Se actualizan los parámetros mediante la función de optimización: $\theta^{t+1} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}(\theta^{(t)})$
5. Se repite este proceso sobre un *dataset* hasta la convergencia de la solución o hasta alcanzar un criterio de parada [**?**].

En resumen, el entrenamiento de la red neuronal es un proceso iterativo de optimización que consiste en:

1. Propagar los datos de entrada por la red.
2. Comparar la salida con el valor real usando una función de perdida.
3. Propagar el error hacia atrás para obtener gradientes.

4. Actualizar los parámetros con un algoritmo de optimización.

5. Repetir hasta que el modelo aproxime adecuadamente la función deseada.

Una vez comprendidas las redes neuronales, podemos retomar la definición de un autómata celular neuronal: una arquitectura que surge de la integración entre los CA y las ANN.

2.4. Autómatas celulares neuronales

En los últimos años, los autómatas celulares neuronales (NCA, *Neural Cellular Automaton*) surgieron como una extensión de los CA clasicos, con las capacidades de aprendizaje y generalización propias de las ANN. Esta integración permite que la regla de transición, tradicionalmente fija, sea parametrizable y entrenable mediante técnicas de aprendizaje supervisado.

Los primeros enfoques imitaban el comportamiento de un CA utilizando una ANN convolucional [8] y fueron popularizados por Mordvintsev et.al. [9], quienes entrenaron un modelo que aprende dinámicas locales a partir de ejemplos. Como resultado, las interacciones locales simples cumplen un objetivo en especifico, mostrando propiedades de crecimiento, regeneración, autoorganización y morfogénesis.

Los NCA son entonces un CA pero cuya función de transición esta definida como una ANN. Retomando la definición previas, podemos decir que:

Definición 18. *Un autómata celular es un autómata celular neuronal si la regla local $\mathcal{R}$ esta dada como:*

$$\mathcal{R} = F_\theta : \mathbb{R}^{|\mathcal{N}|} \to \mathbb{R}^1$$

donde:

- F_θ *es una red neuronal parametrizada por* θ
- $\theta = \{w_{uv}, b_v : (u, v) \in E, v \in V\}$ *es el conjunto de parámetros entrenables.*
- $\mathbb{R}^{|\mathcal{N}|}$ *corresponde al espacio de entrada definido por la vecindad* $\mathcal{N}$
- $\mathbb{R}^1$ *define el valor de salida que representa el nuevo estado de la célula central.*

La figura 2.10 muestra la relación existente entre CA y ANN. En el paso N se tiene una configuración bidimensional con vecindad de Neumann, el estado de cada elemento dentro de la vecindad funciona como valor de entrada para la red neuronal entrenada y devuelve un único resultado, que corresponde a la salida de la célula central, es decir, a la cual se le esta aplicando.

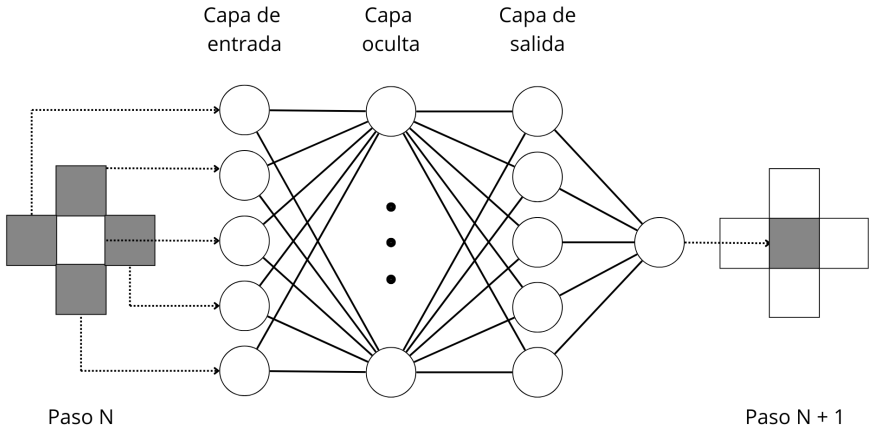


Figura 2.10: Representación grafica de la red neuronal.

En un NCA, cada célula aplica una misma red neuronal local F_θ sobre su vecindad inmediata. La entrada de la red consiste en los estados de las celdas dentro de la vecindad y la salida es el nuevo estado de la célula central.

F_θ suele representarse como una ANN convolucional que permite procesar localmente la información espacial preservando la estructura D-dimensional.

A diferencia de los autómatas clásicos, donde las reglas son explícitas y definidas manualmente, los NCA aprenden sus reglas mediante un proceso de entrenamiento. Se define un conjunto de condiciones iniciales X_0 y un estado objetivo X^*, y la red F_θ se optimiza minimizando una función de pérdida $\mathcal{L}(X_T, X^*)$ que evalúa la diferencia entre la configuración resultante después de T pasos y el objetivo deseado. Este proceso puede expresarse como:

$$\theta^* = \arg\min_\theta \mathcal{L}(F_\theta^{(T)}(X_0), X^*).$$

Durante el entrenamiento, se emplean técnicas de retropropagación a través del tiempo y optimización por gradiente descendente. Esto permite que la red aprenda comportamientos dinámicos complejos, tales como el crecimiento morfológico, la regeneración parcial o la adaptación frente a perturbaciones.

Como se muestra en el siguiente capitulo, los NCA han demostrado ser capaces de generar patrones autoorganizados y adaptativos sin necesidad de una programación explícita de reglas. Algunos ejemplos incluyen:

- **Morfogénesis:** crecimiento de formas y estructuras similares a organismos biológicos.

- **Regeneración:** capacidad de reconstruir regiones dañadas o eliminadas de la configuración.
- **Modelado de sistemas complejos:** simulación de fenómenos físicos, ecológicos o sociales mediante interacciones locales entrenables.

Esta combinación de comportamiento emergente y capacidad de aprendizaje convierte a los NCA en una herramienta poderosa para la exploración de sistemas autoorganizados y adaptativos.

En el transcurso de este marco teórico se abordaron los conceptos fundamentales que sustentan y contextualizan la presente investigación, tales como los sistemas complejos, los autómatas celulares, las redes neuronales y los autómatas celulares neuronales. Estos elementos permiten comprender los principios que rigen la dinámica de sistemas con comportamiento emergente y las herramientas computacionales que posibilitan su modelado y simulación.

La integración de estos fundamentos teóricos proporciona una base sólida para el diseño, desarrollo y validación del simulador propuesto. En particular, la comprensión de la relación entre las reglas locales y los comportamientos globales de los autómatas celulares, junto con la capacidad de aprendizaje de las redes neuronales, constituye el punto de partida para la implementación de autómatas celulares neuronales capaces de adaptarse y evolucionar dentro de entornos complejos.

De esta forma, el marco teórico no solo define el contexto conceptual del trabajo, sino que también sustenta las decisiones técnicas y metodológicas que permitirán la construcción de un modelo de simulación flexible, escalable y coherente con los objetivos planteados en la investigación.

Capítulo 3

Estado del Arte

En 1948, John von Neumann, con ayuda de Stanislaw Ulam, propuso un Autómata Celular (CA, *Cellular Automaton*) como un modelo matemático con la capacidad de autoreproducción universal, el cual desarrollo en su libro *Theory of Self-Reproducing Automata* [1]. Hoy en día representa un modelo fundamental para el estudio de sistemas complejos, computación distribuida y procesos autoorganizados.

Diversos autores, como Stephen Wolfram [3] y Erick W. Weisstein [2] coinciden en caracterizar a los CA como sistemas discretos compuestos por células dispuestos sobre una retícula[1] (*lattice*). Cada celda puede adoptar un valor de un conjunto finito de estados, y su actualización ocurre en pasos discretos de tiempo, determinada por una función de transición que depende únicamente de su vecindad local.

Aunque existen variaciones en la terminología (celda, célula, sitio, color), la estructura formal es ampliamente aceptada y utilizada en campos como la física, biología computacional, procesamiento de imágenes, teoría de la computación, entre otros. Más recientemente, se han integrado técnicas de aprendizaje supervisado, en particular Redes Neuronales Artificiales (ANN), dando lugar a los Autómatas Celulares Neuronales (NCA, *Neural Cellular Automata*).

El presente estado del arte revisa estas propuestas híbridas, enfatizando sus aplicaciones en morfogénesis artificial, clasificación de patrones, generación de texturas dinámicas y modelado en mallas tridimensionales. Además, se identifican

[1] Conjunto de líneas o elementos de estructura lineal que se cruzan o entrelazan estrechamente [10]

retos en términos de escalabilidad, generalización y control de dinámicas emergentes, aspectos que motivan el desarrollo de nuevas variantes de NCA.

3.1. Simulación mediante autómatas celulares

A lo largo del tiempo, se han propuesto múltiples variantes del modelo clásico de autómatas celulares, adaptadas a contextos no estructurados, probabilísticos, heterogéneos o continuos. A continuación, se describen algunas de estas extensiones para el modelado de sistemas complejos.

- Los autómatas celulares en grafos (GCA, *Graph Cellular Automata*) generalizan la estructura reticulada clásica permitiendo vecindades no uniformes: cada célula puede tener un número arbitrario y variable de vecinos. Esta flexibilidad los hace adecuados para modelar fenómenos sobre grafos, como redes sociales o flujos en infraestructura. Sin embargo, diseñar manualmente las reglas de transición en estos entornos es una tarea compleja, lo que ha motivado propuestas como los Graph Neural Cellular Automata (GNCA), que permiten aprender dichas reglas automáticamente [4].
- Los autómatas celulares probabilísticos (PCA, *Probabilistic Cellular Automata*) extienden el modelo clásico al incorporar reglas de transición estocásticas. En lugar de una evolución determinista, los estados futuros dependen de distribuciones de probabilidad condicionadas por la vecindad, lo que permite capturar incertidumbre, ruido y comportamientos emergentes. Son especialmente útiles para simular dinámicas como propagación de epidemias o sistemas sociales, donde los mecanismos de transición no siempre están completamente definidos [5].
- Los autómatas celulares no homogéneos (INCA, *Inhomogeneous Cellular Automata*) permiten que diferentes células tengan diferentes funciones de transición. Para esto, se utilizan dos planos llamados *data plane* el cual contiene la evolución del sistema; y *control plane* el cual define las reglas de transición para cada célula. Esto permite estudiar el comportamiento y la interacción entre diferentes reglas que comparten espacio [6].
- Los autómatas celulares continuos (CCA, *Continuous Cellular Automata*) pueden ser continuos en los estados, en el tiempo y en el espacio [11]:
    - Los estados continuos adquieren valores reales y no solo enteros.

- El tiempo continuo se define mediante una función que determina el intervalo de espera para actualizar la célula en función de su estado actual.
- El espacio continuo se obtiene al asignar diferentes pesos a las células dentro de la vecindad [7].

Si bien estas variantes han expandido el alcance de los CA, un desafío recurrente es la correcta definición de las reglas de transición para reproducir un fenómeno deseado. Este reto ha motivado la introducción de los Autómatas Celulares Neuronales (NCA, *Neural Cellular Automata*), en los cuales la función de transición no se especifica explícitamente, sino que es aprendida mediante una Red Neuronal Artificial (ANN, *Artificial Neural Network*). De este modo, las reglas locales emergen directamente de los datos, lo que facilita modelar dinámicas de alta complejidad.

Un primer avance en esta línea fue presentado por Gilpin [8], demostrando que una Red Neuronal Convolucional (CNN, *Convolutional Neural Network*) puede aprender el comportamiento de un CA clásico. Su estudio destaca que tanto los CA como las CNN comparten dos propiedades fundamentales: (1) la actualización basada en una vecindad o *kernel*, y (2) la aplicación simultánea de dicha función en todo el espacio.

En particular, Gilpin entreno una CNN con un *kernel* de 3×3 (mismo tamaño que la vecindad de Moore) para imitar el comportamiento del *glider*[2] en el Juego de la Vida (GoL, *Game of Life*) de John Conway. Este trabajo estableció una conexión entre los CA y las CNN, mostrando que las ANN puede actuar como mecanismos de transición local aprendidos.

Posteriormente, esta línea de investigación fue extendida por Mordvintsev et al., quienes exploraron representaciones continuas para morfogénesis artificial (*Growing NCA*), y por Niklasson et al., que aplicaron el enfoque a la generación de texturas dinámicas sobre superficies tridimensionales (*Mesh NCA*). A su vez, otros autores han propuesto aplicaciones en robótica, clasificación de patrones y simulación de sistemas fotónicos, consolidando a los NCA como una herramienta para el modelado de sistemas autoorganizados.

[2]Una de las configuraciones mas importantes por su capacidad de moverse en el espacio

3.2. Morfogénesis

Mordvintsev et al. en [9] diseñaron un NCA que permite a una célula crecer para formar un patrón, persistir en el tiempo, regenerarse e incluso rotar. Para esto diseñaron un modelo que depende de un CA continuo con un vector de estados de 16 dimensiones integrando gradientes espaciales mediante filtros de Sobel.



Figura 3.1: Evolución del *emoji* de una lagartija a partir de una única célula activa después de 219 pasos evolutivos [9].

El NCA diseñado tiene las siguientes características (las cuales fueron retomadas por múltiples autores que tomaron a [9] como referencia):

- El vector de cada célula (cada valor en un rango de $[0, 1]$) incluye los canales RGB y un canal α que funciona como umbral para definir si una célula está viva o muerta (establece el resto de valores del vector a 0 si $\alpha \leq 0.1$).
- Considerando las dimensiones (largo, ancho, 16) y un kernel de 3×3 se calcula el gradiente en ambas direcciones (x y y) con un filtro Sobel para producir el nuevo vector que alimentará a la red neuronal:

$$16(\text{estado propio}) + 16(\text{gradiente en } x) + 16(\text{gradiente en } y) = 48$$

- Se usaron convoluciones de 1×1 con una función $ReLU$, alrededor de 8000 parámetros entrenables y devuelve un δ que se suma al estado actual de célula antes del siguiente paso.
- Se omite el cálculo para las células muertas y la actualización de cada célula se realiza con una probabilidad del 50 %. La actualización debe ser estocástica para lograr un mejor rendimiento [12].

Eyvind Niklasson et al. [12] compararon el rendimiento del modelo en [9] con actualización síncrona y asíncrona. Donde, tras haber realizado pruebas con diferentes tamaños de paso, concluyeron que la función de pérdida crece exponencialmente cuando la actualización es síncrona. Sin embargo, esto ocurre únicamente cuando el tamaño de paso es mayor a uno.

Una hipótesis de este comportamiento es que al tener células que se actualizan en diferentes periodos de tiempo, cada célula puede conocer parte del contexto de su vecindad. Esto no se logra únicamente conociendo los valores actuales de cada vecino.

Se realizaron los siguientes experimentos en [9]:

1. Crecer una única célula activa para formar una figura dada mediante *back-propagation*. Cuando él CA alcanza la figura objetivo, al seguir aplicando las reglas de evolución, puede crecer de manera infinita, persistir en la figura, matar a todas las células o generar patrones caóticos o repetitivos.
2. Convertir las figuras objetivo en atractores usando *sample pool*. Se entrenó mediante múltiples condiciones iniciales, obtenidas a partir de entrenamientos fallidos, para llegar al objetivo desde diferentes configuraciones.
3. Regenerar partes de la figura. Se eliminaron partes de la figura objetivo y se añadieron al conjunto de entrenamiento.
4. Rotación de la figura objetivo. Dado que los filtros de Sobel funcionan como direcciones ortogonales, se puede aplicar una rotación mediante:

$$\begin{bmatrix} K_x & K_y \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \cdot \begin{bmatrix} Sobel_x & Sobel_y \end{bmatrix}$$

   para modificar el ángulo sin necesidad de reentrenar el modelo.

3.2.1. NCA Isotrópicos

En [13] [14] [15], Mordvintsev et al. proponen una nueva configuración que no tenga una dirección preferida, la cual nombraron Autómatas Celulares Neuronales Isotrópicos (IsoNCA, *Isotropic Neural Cellular Automata*).

Para volver esta regla isotrópica es necesario que no dependa de la dirección (no distinguir "arriba" de "izquierda"). Esto permite crear rotaciones o simetrías sin que el sistema se descontrole.

Se sustituyen las funciones de los filtros de Sobel en [9] por un único filtro Laplaciano:

$$K_{lap} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & -12 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

el cual se concatena al vector de cada célula y alimenta a una ANN de 9408 parámetros.

A continuación, se describen los resultados obtenidos con dos diferentes enfoques:

1. Estrategia de *seed* (semilla) estructurada: Es necesario de (al menos) tres células con valores diferentes no colineales para definir la orientación y la posición. Modificar la posición de estos puntos implica una evolución acorde a la posición de los mismos.

2. Estrategia de una única *seed*: Se modifica la función de pérdida para que sea invariante ante rotaciones y reflejos. Para evitar comparar la figura resultante con todas las rotaciones posibles, se mapean ambas figuras a coordenadas polares y, para cada posible ángulo de rotación, se calcula la diferencia. Para disminuir la complejidad computacional, se aplica la transformada rápida de Fourier y haciendo uso del teorema de la convolución:

   $$s * t = \mathcal{F}^{-1}(\mathcal{F}(S) \cdot \mathcal{F}(T))$$

   se obtienen todas las diferencias únicamente como una multiplicación de transformadas.

   Para evitar que se creen patrones simétricos mezclados (Figuras con cabeza-cabeza o cola-cola) se añade un nuevo canal llamado *binary auxiliar channel* que asigna valores de -0.5 y 0.5 a la parte superior e inferior de la imagen.

   Con estas nuevas configuraciones se puede obtener el mismo resultado que en [9] pero sin la dependencia de una dirección explícita y con una ANN con menos parámetros.

3.2.2. Autoclasificación

Mordvintsev et al. en [16] utilizan los NCA para la autoclasificación de MNIST[3] las cuales fueron binarizadas con un umbral de 0.1.

El objetivo es que, dada una configuración inicial de células vivas, el sistema sea capaz de identificar la forma únicamente mediante reglas locales, en este caso, que pueda identificar los números de MNIST.

Estos son los cambios de *MNIST CA* respecto al modelo original [9]:

[3]Conjunto de datos de imágenes 28×28 de números en el rango $[0, 9]$

- Vector de la célula: Originalmente se usaban los canales RGB, ahora se usan 10 canales que funcionan como distribución de probabilidad de cada posible número. Durante la evaluación se utiliza el mayor de ellos.
- Célula viva o muerta: en [9] se define mediante el canal α, pero las células muertas siguen siendo visibles para el resto. En [16] únicamente interactúan las células vivas.
- Topología: Todas las células son una única componente conexa.

Con el nuevo modelo, Mordvintsev et al. pudieron realizar clasificaciones con una precisión del 95.3 % con una función de pérdida L_2 y ruido añadido. Este experimento demostró que los NCA no solo pueden generar patrones y autorregenerarse, sino también realizar tareas de clasificación distribuidas sin una capa de decisión global.

Aunque los modelos *Growing CA* y *MNIST CA* son robustos frente a diferentes tipos de perturbaciones, su comportamiento se puede ver afectado por: estados individuales de las células, parámetros del modelo y el campo perceptivo (la manera en como las células perciben su vecindad).

Es por esto que Randazzo, et al. [17] buscan enfrentar estrategias adversarias en el mismo espacio celular para estudiar la reprogramación de los CA mediante dos experimentos:

1. Inyección de adversarios: Se entrena un nuevo modelo de CA, que se coloca en el entorno que ejecuta uno de los modelos originales, para secuestrar el comportamiento del colectivo.
2. Perturbación de estados: A un modelo ya entrenado, se le aplican perturbaciones a todas las células del sistema de forma simultánea.

Para la inyección de adversarios se utilizan dos modelos: el modelo original *MNIST CA* con probabilidad de 90 % y un nuevo modelo que clasifica a todos los números como “ocho” (10 %). Se “inyectan” múltiples células invasoras en figuras ya clasificadas las cuales muestran un comportamiento dominante: clasificando a todos los elementos como “ocho”. Además, una vez retiradas las células invasoras, el sistema es capaz de recuperarse la mayoría de las veces.

Sin embargo, al aplicar el mismo procedimiento para *Growing CA* los resultados no tuvieron el mismo rendimiento. Probaron modificar la morfología de la imagen

de una lagartija sin cola y con otro color, pero ninguna estrategia fue capaz de invadir por completo el modelo original.

Una hipótesis es que *MNIST CA* no necesita reconfigurar todo su valor para generar algo distinto a lo que ya generaba antes, pues el estado de la mutación es un estado ya considerado en el modelo original. Por otro lado, al cambiar el color y la forma en *Growing CA* se están explorando configuraciones nunca antes vistas por el modelo original.

Para el segundo experimento (perturbación de estados), se sigue la línea de modificar el color y la forma de la imagen de una lagartija. Se aplica una perturbación global de estados cada paso de tiempo con el objetivo de identificar la perturbación específica que resulte en un patrón estable.

En cada paso evolutivo se multiplica el vector de cada célula por una matriz simétrica A de 16×16 [9]. Para definir los valores de la matriz se inicia con una matriz simétrica I y, a diferencia de *Growing CA*, la configuración inicial se establece como la figura objetivo del modelo original.

Esta configuración permite crecer un sistema con perturbaciones, alterar su forma y color una vez crecido y modificar el impacto y el número de perturbaciones posibles. Al manejar matrices simétricas se puede representar como

$$A = Q\Lambda Q^T$$

donde:

- Λ es la matriz diagonal de los valores propios.
- Q es la matriz unitaria de los vectores propios.

Si $\Lambda = I \rightarrow A = I$ por lo que no existe perturbación (multiplicación por matriz identidad). Bajo esta premisa, se propone la siguiente fórmula:

$$A_k = Q(kD + I)Q^T$$

donde:

- A es la matriz original.
- k es el factor de escala.
- D es la dirección de la perturbación $\Lambda - I$ en el espacio de valores propios.

Si $k = 0$ no existe perturbación en el sistema:

$$A_k = Q(0D + I)Q^T$$
$$A_k = QIQ^T$$

Si $k = 1$ se conserva la perturbación original:

$$A_k = Q(1D + I)Q^T$$
$$A_k = Q(\Lambda - I + I)Q^T$$
$$A_k = Q\Lambda Q^T$$

Mostrando que se puede escalar el tamaño de la perturbación, incluso con perturbaciones inversas con $k = -1$.

Si consideramos dos perturbaciones cuyos vectores propios son similares: $A_k = Q(k_A D_A + I)Q^T$ y $B_k = Q(k_B D_B + I)Q^T$ podemos combinarlos como:

$$comb(A_k, B_k) = Q(k_A D_A + k_B D_B + I)Q^T = k_A A + k_B B + (1 - k_A - k_B)I$$

Esta nueva combinación permite configurar múltiples perturbaciones, las cuales se vuelven más sensibles conforme aumentan los k.

Estos experimentos muestran como los NCA pueden ser vulnerables a diferentes estrategias y como las mutaciones "exactas" pueden modificar la estructura de todo el sistema, mostrando un comportamiento similar a lo que ocurre en teoría de juegos evolutivos.

Todas estas representaciones han tenido un lugar en un espacio bidimensional (2D). A continuación se presentan algunas propuestas en espacios tridimensionales (3D).

3.2.3. Representaciones tridimensionales

Zhang et al. [18] proponen un modelo generativo tridimensional denominado *Generative Cellular Automata* (GCA), que extiende el paradigma de los CA hacia la generación progresiva de formas 3D. A diferencia de enfoques que intentan reconstruir directamente la forma completa, GCA formula el proceso como una cadena de Markov, donde cada paso corresponde a una transición local sobre una rejilla tridimensional.

El modelo se entrena para que estas reglas locales, al repetirse, converjan hacia formas específicas. Para ello, se representa la forma como puntos de superficie en

una rejilla 3D y se emplea una Red Convolucional Dispersa (*Sparse CNN*) que opera únicamente sobre las regiones activas.

Durante el entrenamiento, se utilizan secuencias ligeramente distintas a las cadenas de muestreo reales, pero que convergen hacia las formas del conjunto de datos. Esto permite que el modelo aprenda reglas de transición robustas, capaces de generar formas diversas desde configuraciones parciales. A través de una sola inferencia, el sistema puede muestrear instancias variadas de la distribución aprendida, logrando resultados competitivos en tareas de completado y generación de formas 3D.

Este enfoque representa una contribución relevante al campo de la morfogénesis computacional, al demostrar que las reglas locales autoorganizadas pueden escalar hacia dominios 3D complejos mediante arquitecturas neuronales adaptadas.

En [18] demuestran que la autoorganización local escala a generación 3D, por otro lado, Shyam Sudhakaran et al. [19] expanden el trabajo de [9] a tres dimensiones utilizando como espacio celular el juego de Minecraft[4]. El NCA entrenado puede construir estructuras de hasta 3584 bloques con 50 bloques únicos conservando propiedades regenerativas de su contraparte en 2D.

Una entidad en Minecraft se representa como una red 3D donde cada elemento es un vector que almacena (1) el tipo de bloque (incrustado en un vector *one-hot*), (2) su estatus (replicando el comportamiento del canal α de [9]) y (3) estados ocultos.

Se utilizan convoluciones para permitir la generación y manipulación de estructuras 3D. Además, se sustituye el uso de filtros de Sobel por una red de percepción aprendible con un kernel de $3 \times 3 \times 3$, *stride*[5] de 1 y canales de salida equivalentes a tres veces el número de canales del estado celular. Esta red permite que el sistema aprenda activamente a percibir su entorno local, adaptando sus filtros durante el entrenamiento y capturando patrones espaciales relevantes de manera dinámica.

Se utiliza una actualización estocástica y un umbral de activación similar a [9]. Finalmente, todos los modelos se inicializan con una distribución normal estándar, con media cero y desviación estándar en el rango de 0.0001 a 0.1, lo que proporciona un punto de partida estable para el entrenamiento y la evolución del sistema. Este sistema hereda las propiedades regenerativas de su versión 2D.

Finalmente, Horibe et al. [20] lleva las propiedades regenerativas al mundo físi-

[4]Permite crear estructuras estáticas (como árboles) y estructuras dinámicas (como máquinas voladoras)

[5]Cuántas posiciones se desplaza el *kernel* sobre la entrada en cada operación de convolución

co. Se implementa el enfoque regenerativo en robots blandos mediante NCA. Cada robot está definido por un arreglo de volúmenes de reposo para cada vóxel[6], que se activan mediante una señal global sinusoidal, provocando expansión y contracción volumétrica. Esta dinámica permite simular locomoción a través de ciclos de actuación, evaluando el desplazamiento del centro de masa como métrica de fitness. Los robots blandos se componen de distintos tipos de vóxeles: músculos en fase y contra-fase, huesos rígidos y flexibles, cada uno con un rol en la locomoción.

El modelo entrenado demuestra que los robots blandos pueden recuperar parcialmente su morfología original tras sufrir daño, y restaurar su capacidad de movimiento sin intervención externa. Este comportamiento emerge de la dinámica autoorganizada del NCA, que codifica implícitamente la forma objetivo y su capacidad de reparación.

Para evaluar la capacidad de locomoción y regeneración de los robots blandos generados mediante NCA, se emplea el simulador físico Voxelyze. Los experimentos se realizan en configuraciones 2D y 3D, utilizando rejillas de 7×7 y $9 \times 9 \times 9$ vóxeles respectivamente, con condiciones de frontera tipo Dirichlet que limitan el crecimiento morfológico a regiones internas (5×5 en 2D y $7 \times 7 \times 7$ en 3D).

Este entorno permite validar la capacidad autoorganizada de los NCA para generar morfologías funcionales y resilientes. Ademas, proporciona una base cuantitativa para comparar la eficiencia locomotora antes y después de procesos de regeneración. La estructura voxelizada facilita la integración de reglas locales y codificación distribuida, alineándose con marcos bioinspirados de morfogénesis computacional.

En conjunto, estos trabajos evidencian cómo los NCA pueden extender sus principios regenerativos y de autoorganización a dominios tridimensionales cada vez más complejos. Mientras GCA establece un marco generativo para formas 3D a partir de reglas locales aprendidas, los experimentos en Minecraft muestran que dichas propiedades pueden preservarse incluso en entornos con alta variabilidad estructural. Finalmente, la aplicación en robots blandos confirma el potencial bioinspirado de los NCA: no solo permiten generar morfologías funcionales, sino que además dotan a los sistemas físicos de resiliencia ante daños, abriendo camino hacia formas de robótica autoreparable.

Otra forma de implementar los NCA en 3D es mediante planos y texturas autoorganizadas que cubren la superficie de un objeto tridimensional.

[6]Unidad básica de un espacio tridimensional, similar a un píxel en imágenes 2D, pero con volumen

3.3. Mesh Neural Cellular Automata

Niklasson et al. [21] y Mordvintsev et al. [22] [23] introducen el concepto de *Self-Organising Textures*, cuyo objetivo es reproducir la apariencia global de una textura de referencia sin generar copias exactas. Para ello, entrenan un NCA con la misma arquitectura neuronal propuesta en [9], de modo que pueda capturar y generar patrones característicos como tableros de ajedrez, líneas o mosaicos. Aunque este enfoque tiene un carácter demostrativo, constituye una evidencia importante de que los NCA pueden aprender y generalizar comportamientos visuales locales en lugar de memorizar imágenes específicas.

Sobre esta base, Niklasson et al. proponen *Mesh Neural Cellular Automata* (MeshNCA) [24], un método diseñado para generar texturas dinámicas en superficies tridimensionales. A diferencia de los experimentos en cuadrículas bidimensionales, MeshNCA extiende el mecanismo de autoorganización a mallas arbitrarias, incluso cuando estas se encuentran en movimiento. El modelo mantiene coherencia espacial en distintos escenarios, muestra robustez frente a perturbaciones externas y permite controlar propiedades como densidad, dirección y rotación de los patrones emergentes. Además, varios MeshNCA pueden injertarse de manera conjunta, produciendo transiciones suaves entre texturas sin discontinuidades.

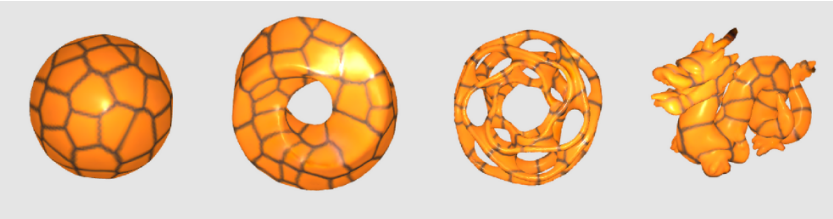


Figura 3.2: El entrenamiento se realiza sobre un poliedro simétrico y el NCA puede ser extrapolado a cualquier objeto tridimensional [9].

Sin embargo, este enfoque presenta un reto de escalabilidad. El número de células involucradas suele encontrarse en el rango de $10^4 \sim 10^5$, lo que limita la resolución alcanzable a entre 64×64 y 256×256 píxeles. Como el tiempo de entrenamiento escala de forma cuadrática con respecto al número de celdas, el aumento de resolución conlleva un cuello de botella en la propagación de información a lo largo de la malla.

Para superar esta limitación, Niklasson et al. proponen en [25] un modelo híbrido que combina NCA con Representaciones Neuronales Implícitas (INR, *Implicit Neural Representations*). La idea central es desacoplar el tamaño de la cuadrícula del NCA de la resolución final, delegando el renderizado detallado a un decodifica-

dor ligero denominado *Local Pattern Producing Network* (LPPN). De este modo, el NCA mantiene una cuadrícula explícita y relativamente pequeña que evoluciona de manera local, mientras que la LPPN se encarga de producir el color en cualquier punto del espacio continuo a partir de información interpolada de las celdas vecinas.

En términos formales, para cada punto de muestreo p, la LPPN recibe dos entradas: (1) un estado celular promedio $s(p)$, obtenido mediante interpolación de las celdas cercanas en un primitivo geométrico, y (2) un vector de coordenadas locales $u(p)$, que codifica la posición relativa de p dentro de dicho primitivo. Dependiendo de la discretización empleada, estos primitivos pueden ser rectángulos, cubos, triángulos o polígonos generales, definidos por coordenadas $\lambda_i(p)$ que cumplen propiedades como unidad de partición y no negatividad. Para garantizar continuidad espacial, las coordenadas se transforman mediante funciones sinusoidales en cuadrículas cartesianas, o bien en mallas triangulares, obteniendo así campos posicionales suaves.

El resultado es un sistema que conserva las propiedades autoorganizativas de los NCA al tiempo que permite alcanzar mayor resolución sin un aumento proporcional del costo computacional. Desde una perspectiva biológica, el NCA puede interpretarse como un plano morfogenético que define la organización global, mientras que la LPPN añade los detalles finos sin necesidad de simular interacciones locales a gran escala. Este paralelismo, aunque principalmente ilustrativo, refuerza la idea de que los NCA pueden actuar como generadores de estructuras robustas y adaptables, mientras que módulos complementarios aportan precisión y escalabilidad.

A pesar de estas mejoras, una limitación persistente es que cada NCA debe entrenarse de manera específica para una textura o patrón dado. Incluso en escenarios donde múltiples NCA coexisten, cada modelo requiere su propio entrenamiento independiente. Esto ha motivado nuevas propuestas orientadas a la construcción de espacios compartidos de NCA, capaces de generalizar a múltiples texturas dentro de un mismo marco de entrenamiento.

3.4. Modelos generativos

A cada imagen generada con un NCA [9] le corresponde una ANN que está explícitamente entrenada para imitar ese comportamiento. Alejandro Hernández et al. [26] proponen una ANN de mayor capacidad que permita crecer múltiples imágenes, a lo que llamaron *Neural Cellular Automata Manifold* (NCAM).

El NCAM usa un modelo de convoluciones dinámicas para modelar un espacio de NCAs. Utiliza un auto-*encoder* que se compone de un codificador y un decodificador: El codificador transforma la entrada en una representación continua y luego categórica, que el decodificador reconstruye en el espacio original.

Un elemento destacable de la arquitectura propuesta es un parámetro entrenable llamado *Leak Factor* (LF) que mejora la estabilidad durante el entrenamiento. Este parámetro regula la cantidad de información que se filtra en cada paso del modelo, por ejemplo, un valor bajo retiene el conocimiento acumulado. Este valor se incrementa durante el entrenamiento a medida que mejora el rendimiento.

La arquitectura también emplea convoluciones dinámicas. Los *kernels* se generan a partir del *embedding* (incrustación) de cada instancia, permitiendo adaptabilidad sin modificar la estructura del NCA. El NCAM inicia desde una imagen *seed* y actualiza cada celda mediante percepciones locales (filtros Sobel + normalización) y capas densas por píxel, moduladas por el LF. Se exploran actualizaciones estocásticas y sincrónicas, siendo esta última más precisa y estable, tal como en [12].

Los experimentos confirman que la codificación y la actualización sincrónica mejoran la calidad de reconstrucción, incluso en ausencia de mecanismos de enmascaramiento. También se observa que aumentar la dimensionalidad del *embedding* mejora significativamente los resultados.

Por otro lado, Kevin Frans en [27] propone un modelo llamado *Conditional Neural Cellular Automata* (StampCA) cuya *seed* recibe un *embedding* el cual define la imagen en la cual va a crecer.

Dado un diseño deseado, una red de codificador lo comprime en un *embedding* el cual se utiliza como condición inicial en la NCA, que luego itera para crecer en la figura objetivo, esto permite crear nuevas configuraciones

Utilizando un *dataset*(conjunto de datos) con aproximadamente 3000 *emojis* de 32×32 píxeles fue capaz de entrenar a StampCA utilizando el mismo método de [9] para poder dibujar todos los *emojis* con una única NCA.

Para generar números de MNIST se adopta un enfoque tipo *Generative Adversarial Network* (GAN), donde el objetivo es crecer dígitos desde ruido aleatorio. Dado que entrenar directamente un NCA como generador GAN resultó inestable, se utilizó una estrategia en dos etapas: (1) Se entrena primero un generador tradicional (*feedforward*) para mapear vectores aleatorios a dígitos MNIST y (2) el NCA se entrena para imitar la salida de ese generador, logrando una generación más estable.

Este enfoque permite que el NCA genere dígitos sin necesidad de un *dataset* explícito, aprendiendo directamente a partir del objetivo GAN. Además, se observa un patrón de crecimiento distintivo: mientras los *emojis* crecen en ondas desde el centro, los dígitos MNIST se desarrollan siguiendo la forma del trazo.

Mingxiang Chen et al. [28] combinan los principios de *Variational Auto-Encoder* (VAE) con NCA. Utilizan un *encoder* que transforma una imagen en una distribución de probabilidad, las cuales forman un *embedding* el cual funciona como condición inicial del NCA. Posteriormente Rasmus Berg Palm et al. [29] nombran a este modelo como *Variational Neural Cellular Automata* (VNCA), quienes ademas aseguran que ninguno de los otros modelos vistos ([9], [26], [18], [20] y [27]) en realmente generativo.

El VNCA se basa en la arquitectura del modelo generativo probabilístico *Variational Auto-Encoder* (VAE) la cual aprende una distribución de probabilidad que codifica los datos observados. A partir de esta representación, el decodificador genera imágenes mediante un NCA que simula el crecimiento espacial desde una *seed*. A diferencia de VAE que decodifica la imagen directamente, el VNCA utiliza el *embedding* como configuración inicial de una cuadricula de celdas que evolucionan siguiendo las reglas definidas por una CNN.

Por otra parte, este modelo incorpora mecanismos de duplicación celular inspirado en la mitosis. Cada M pasos las celdas se duplican hacia la derecha y hacia abajo permitiendo a la imagen crecer de manera autoorganizada.

Los experimentos realizados en [29] demuestran que el VNCA es capaz de aprender procesos generativos autoorganizados tanto en dominios simples como MNIST, como en conjuntos más complejos como CelebA[7]. En MNIST, el modelo logra reconstrucciones precisas y muestras plausibles desde el espacio latente, con una log-verosimilitud de $\log p(x) \geq -84.23$ nats[8], su rendimiento es comparable a modelos autorregresivos como PixelCNN. Además, el análisis del espacio latente revela una estructura más separable y organizada que la de decodificadores convolucionales tradicionales, lo que sugiere una codificación más semántica y eficiente.

En CelebA, aunque las muestras generadas directamente no alcanzan calidad visual comparable a modelos de última generación, el VNCA muestra capacidad de reconstrucción robusta y mejora visual significativa al ajustar el parámetro β, que regula el equilibrio entre fidelidad de reconstrucción y calidad de generación.

[7] *Dataset* de imágenes de celebridades

[8] Unidad de medida utilizada en aprendizaje automático y estadística para expresar la log-verosimilitud o la entropía de un modelo

Finalmente, se explora la resiliencia del VNCA ante daño estructural durante el crecimiento. Al eliminar las capas de duplicación y entrenar con un esquema de recuperación de daño, el modelo aprende dinámicas estables capaces de restaurar imágenes deterioradas casi por completo, incluso tras múltiples pasos de evolución. Este comportamiento sugiere que el VNCA no solo codifica la imagen, sino que también es un proceso de reconstrucción en el estado celular.

3.5. Representaciones y aplicaciones adicionales

Hasta el momento, gran parte de los trabajos con NCA se han centrado en la morfogénesis y en su capacidad para generar estructuras visuales autoorganizadas. Sin embargo, también han surgido enfoques que amplían el marco conceptual hacia nuevas formas de representación y aplicaciones en dominios específicos. A continuación, se presentan algunas de estas variantes.

Mattie Tesfaldet et al. [30] extienden los NCA mediante mecanismos de atención inspirados en arquitecturas de *transformers*, dando lugar a *Vision Transformer Cellular Automata* (ViTCA). Esta arquitectura híbrida permite que las interacciones locales tengan acceso contextual a regiones más amplias a través de *self-attention*, lo que habilita organización global sin incurrir en la complejidad cuadrática típica de la atención completa. ViTCA adapta el proceso de los ViT tradicionales, que dividen una imagen en parches y los convierten en secuencias de *tokens*, al formato de una cuadrícula de celdas, donde cada celda almacena información visual, posicional y oculta. En lugar de aplicar atención global, restringe el cálculo a vecindarios locales, manteniendo así la naturaleza distribuida del sistema y reduciendo el costo computacional. El entrenamiento se realiza de forma iterativa, alternando entre inyectar entradas corruptas y actualizar parcialmente los estados celulares hasta reconstruir la imagen. De este modo, ViTCA combina eficiencia, capacidad de autoorganización y organización espacial coherente.

En un ámbito distinto, Jian Tang et al. [31] emplean CNN para simular el desarrollo de microestructuras en la solidificación de metales. Cada celda del dominio contiene un estado de fase $P \in [0, 1]$ (donde $p = 0$ representa líquido y $p = 1$ sólido), tres ángulos de Euler $\theta = (\alpha_1, \alpha_2, \alpha_3)$ que definen su orientación cristalográfica, y la magnitud de sub enfriamiento $\Delta T = T_{melting} - T$. La evolución se modela con un NCA que aplica recurrentemente una CNN para predecir los incrementos en fase, orientación y estados ocultos. Estos canales ocultos facilitan el aprendizaje de patrones temporales complejos sin necesidad de reglas explícitas. Con ello, el modelo reproduce fenómenos como crecimiento competitivo o direc-

ciones preferentes, con una eficiencia computacional sustancialmente mayor que la de muchos CA tradicionales y con capacidad de generalización fuera del rango de entrenamiento.

Por otro lado, Gordon H. Y. Li et al. [32] proponen el modelo *Photonic Neural Cellular Automata* (PNCA), que aprovecha la velocidad y la interconexión masiva de la fotónica para implementar NCA en hardware óptico. A diferencia de arquitecturas convencionales basadas en MLP o CNN, difíciles de trasladar a entornos fotónicos por sus conexiones densas, PNCA utiliza reglas locales aprendidas mediante técnicas de aprendizaje profundo. Esto permite realizar clasificación de imágenes en sistemas ópticos, donde la actualización celular se implementa mediante interferencia lineal de luz y activaciones ópticas no lineales. Los autores demuestran clasificación binaria con alta precisión utilizando un número muy reducido de parámetros fotónicos programables, mostrando además robustez frente a datos fuera de distribución. PNCA representa una vía prometedora para llevar el aprendizaje autoorganizado a plataformas fotónicas escalables.

En el ámbito de la biología computacional, Caitlin Grasso y Josh Bongard [33] introducen los *Empowered Neural Cellular Automata* (ENCA), una extensión que incorpora el concepto de *empowerment* —la cantidad de control que una célula puede ejercer sobre su entorno— formalizado mediante medidas de información mutua entre acciones pasadas y estados sensoriales futuros. Incluir *empowerment* como objetivo adicional en tareas de morfogénesis mejora la aptitud evolutiva y produce patrones de crecimiento más robustos y adaptativos, posicionando a cada célula como un agente autónomo dentro de un sistema distribuido y abriendo vías para estudiar coordinación y comunicación celular desde una óptica computacional.

Finalmente, mientras que modelos como ViTCA, PNCA o ENCA amplían las capacidades arquitectónicas y las plataformas de aplicación de los NCA, Richardson et al. [34] utilizan los NCA como herramienta para modelar fenómenos dinámicos complejos. Entrenando NCA mediante optimización basada en gradientes para reproducir trayectorias espacio-temporales generadas por ecuaciones diferenciales parciales (por ejemplo, Gray–Scott), este enfoque captura tanto estructuras transitorias como patrones estacionarios —incluyendo la formación de patrones tipo Turing— y aborda el problema inverso de inferir reglas locales a partir de comportamiento emergente. Con una complejidad de red mínima, los autores muestran capacidad de generalización, respeto por simetrías y robustez frente a perturbaciones, posicionando a los NCA como un marco explicativo y eficiente para sistemas biológicos y físicos.

Se observa que los NCA han evolucionado desde sus primeras aplicaciones en morfogénesis, donde se exploró su capacidad de generar estructuras complejas par-

tiendo de una única célula activa, hacia modelos que buscan una mayor versatilidad generativa, capaces de producir dinámicas visuales, texturas, comportamientos adaptativos e incluso manipular estrategias, muchos de ellos inspirados en la definición de Mordvintsev. Además, se han desarrollado arquitecturas híbridas que no generan un NCA individual, sino que modelan un espacio de NCA, así como configuraciones generativas y bioinspiradas.

En conjunto, estos trabajos evidencian que los NCA no solo constituyen un marco computacional de interés teórico, sino también una herramienta con aplicaciones en inteligencia artificial, ciencias naturales e ingeniería. Esta diversidad de enfoques abre la puerta a explorar nuevas adaptaciones y propuestas, especialmente en áreas de aprendizaje profundo, autoorganización y simulación de procesos dinámicos.

Capítulo 4

Marco metodológico

El marco metodológico se constituye como el resultado de la aplicación sistemática y lógica de los conceptos y fundamentos abordados en el marco teórico. Azuero et al. [**?**] lo definen como un conjunto de acciones destinadas a describir y analizar el fondo del problema planteado, mediante procedimientos específicos que incluyen técnicas de observación y recolección de datos, los cuales determinarán el cómo se realizará la investigación.

Este capítulo se dividirá en secciones que definan la estructura del marco metodológico, con el fin de sustentar los métodos y recursos elegidos para el desarrollo de la investigación, asegurando que cada fase esté alineada con los principios teóricos fundamentales y que los resultados obtenidos sean fiables, replicables y efectivos en el tratamiento de la problemática planteada.

4.1. Diseño de la investigación

4.1.1. Diseño experimental

El diseño experimental es un método sistemático para planificar, ejecutar y analizar experimentos, con el objetivo de identificar relaciones causales entre variables. En otras palabras se manipula una variable independiente para observar su efecto sobre la variable dependiente [**?**].

De acuerdo con su nivel de control, los diseños experimentales se clasifican en tres tipos: pre-experimentos, cuasi-experimentos y experimentos puros. En el

presente trabajo se adopta un diseño cuasi-experimental por las características propias del entorno de simulación y la naturaleza del objeto de estudio.

El diseño cuasi-experimental resulta apropiado en este contexto, ya que se manipula una variable independiente (la definición de la función de transición mediante una red neuronal) para observar sus efectos sobre el comportamiento del sistema simulado, sin que exista una asignación aleatoria ni un grupo control en un sentido estricto.

4.1.1.1. Diseño cuasi-experimental

El diseño cuasi-experimental busca establecer relaciones causales sin la necesidad de asignar aleatoriamente a los participantes o unidades experimentales a los grupos de estudio. Se caracteriza por la existencia de una intervención deliberada sobre una variable independiente, mientras que los grupos de comparación se definen con base en condiciones preexistentes [**?**].

En el caso de esta investigación, el estudio se desarrolla en un entorno computacional controlado, donde las condiciones experimentales se reproducen mediante simulaciones. El objetivo principal consiste en analizar la influencia de distintos parámetros, como la estructura de la red neuronal o la configuración del autómata, sobre los patrones emergentes generados por los autómatas celulares neuronales. Este enfoque permite observar efectos y comportamientos resultantes, manteniendo un control suficiente sobre las variables involucradas sin requerir un esquema de aleatorización.

4.2. Enfoque de la investigación

El presente proyecto tiene un enfoque cuantitativo, el cual se caracteriza por la recolección y el análisis de datos numéricos para poder describir fenómenos mediante procedimientos estadísticos y matemáticos [**?**]. Este enfoque prioriza la objetividad y medición precisa para relacionar causales entre las variables observadas.

A diferencia de los enfoques cualitativos o mixtos, este no requiere de estudiar percepciones, significados ni experiencias humanas, por lo que es adecuado para investigaciones donde las variables pueden ser definidas, controladas y medidas [**?**].

El enfoque cuantitativo se justifica porque el análisis se realizar a partir de datos

generados mediante simulación computacional. En este proceso no hay intervención directa con humanos y los resultados obtenidos no están sujetos a interpretaciones. Este enfoque permite comparar escenarios, desempeño de modelos e influencia de parámetros sobre la dinámica emergente del sistema.

4.3. Población

La población de estudio se define como un conjunto cerrado, finito y accesible de casos (personas, animales, expedientes, objetos, etc.) los cuales cumplen con un criterio de inclusión y que sirven como base para la investigación [**?**]. Es otras palabras, es un conjunto conformado por todos los elementos a los que se pretende generalizar los resultados del estudio.

En el presente trabajo, la población de estudio esta conformada por todos aquellos autómatas celulares finitos localizados en un espacio bidimensional con una distribución cuadriculada regular. Se consideran condiciones de frontera un conjunto de estados finito y cualquier función que pueda mapear de $\mathbb{R}^{|\mathcal{N}|} \to \mathbb{R}$.

4.4. Muestra y muestreo

La muestra corresponde a un subconjunto de la población de estudio y consiste en el numero de casos que realmente serán estudiados. Esta debe ser representativa para garantizar la validez de los resultados. Por otro lado, el muestreos e refiere al proceso aplicado por el investigador para seleccionar a los participantes dentro de la población [**?**].

Existe el muestreo probabilístico (donde cada miembro tiene la población tiene una probabilidad de ser escogido) y no probabilístico (donde la selección se basa en la conveniencia, el juicio del investigador u otros criterios) [**?**].

Para esta investigación se realiza un muestreo no probabilístico intencional, donde los elementos seleccionados para la muestra son elegidos por el criterio del investigador. Esto se justifica ya que, al existir una infinitud de autómatas celulares dadas las restricciones en la población, es necesario delimitar y tratar de replicar de los comportamientos. Ademas, dado que cada CA resuelve un problema, es necesario identificar de forma precisa el problema que se esta resolviendo [**?**].

4.5. Tipo de investigación

Con base en los objetivos, esta investigación se considera aplicada. La investigación aplicada, también conocida como práctica o empírica, se caracteriza por porque busca aplicar o utilizar los conocimientos adquiridos. En este tipo, las consecuencias practicas son un interés primordial del investigador [**?**].

Ademas, se adopta un diseño cuasi-experimental, dado que se manipulan variables independientes para observar su efecto sobre los resultados de la simulación, sin la existencia de un grupo control o asignación aleatoria de condiciones experimentales.

Este tipo de investigación resulta adecuado, ya que permite establecer relaciones causales entre parámetros del modelo y comportamientos emergentes del sistema, manteniendo al mismo tiempo un entorno computacional controlado y reproducible.

4.6. Técnicas e instrumentos de recolección de datos

La recolección de datos se realiza a partir de las simulaciones computacionales realizadas. Dado que se trata de enfoque cuantitativo, se es necesario usar técnicas de observación como encuestas, sino procedimientos de registro automatizado para su posterior análisis numérico.

4.6.1. Técnicas de recolección de datos

- Simulación computacional: Ejecución del modelo del automata celular baja diferentes configuraciones de vecindad, estados y arquitectura de ANN.
- Registro automatico de datos: Datos generados por la simulación. Como el historial evolutivo, patrones, y métricas de estabilidad, error y desempeño.
- Análisis estadístico: Se aplican métricas cuantitativas que permiten comparar configuraciones para evaluar la efectividad del modelo. Métricas como precisión, convergencia, tasa de error y comportamiento temporal de los patrones.

4.6.2. Instrumentos de recolección de datos

Se consideran los siguientes instrumentos para obtener y analizar los datos:

- Simulador desarrollado: Herramienta computacional implementada en Python que permite ejecutar y visualizar el comportamiento de los autómatas celulares neuronales.
- Códigos de registro y procesamiento: Conjunto de programas auxiliares que capturan las métricas numéricas de cada simulación y almacenadas en archivos.
- Herramientas de análisis estadístico y visualización: Librerías y herramientas que permiten obtener representaciones graficas con los datos obtenidos.

4.7. Procedimiento

El desarrollo metodológico de la investigación se llevará a cabo en diversas fases secuenciales. Estas abarcan desde el diseño e implementación del simulador hasta la evaluación del modelo de NCA. Cada fase esta orientada a cumplir los objetivos planteados y a generar datos cuantitativos que permitan analizar el comportamiento de los sistemas simulados.

- **Diseño, desarrollo e implementación del simulador**: Se construye una herramienta computacional capaz de simular CA bidimensionales. Aunque el fin ultimo es estudiar los NCA, este simulador debe de poder aceptar configuraciones clásicas. Esto debido a que, para las siguientes fases, es necesario conocer el comportamiento de configuraciones conocidas.
- **Diseño, desarrollo e integración del módulo ANN**: Se implementa un módulo adicional que permite entrenar ANN para aprender el comportamiento de autómatas celulares existentes. La salida de este modulo, es la función de transición aprendida. Ademas, este módulo se integra dentro del simulador principal para permitir la experimentación directa.
- **Imitación de patrones conocidos**: Se diseñan y ejecutan experimentos para evaluar la capacidad del modelo neuronal de imitar el comportamiento de autómatas celulares clásicos, como GoL. Se busca ademas, medir la precisión del aprendizaje y la estabilidad de las configuraciones generadas.

- **Simulación de otros sistemas complejos:** Una vez validado el funcionamiento del modelo neuronal, se aplican las mismas técnicas a simulaciones que representen otros sistemas complejos. Se busca probar diferentes configuraciones iniciales y parámetros para analizar la capacidad de generalizar la aparición de nuevos patrones.

Cada una de estas fases produce resultados intermedios que retroalimentan el proceso de desarrollo, permitiendo refinar tanto la arquitectura de la red neuronal como las condiciones experimentales del simulador.

4.8. Técnicas y herramientas de procesamiento de datos

Una vez recolectados los datos provenientes de las simulaciones computacionales, es fundamental aplicar técnicas adecuadas de procesamiento y análisis para garantizar la validez de los resultados y su correcta interpretación. Dado que el enfoque de la investigación es cuantitativo, el procesamiento se centra en el análisis numérico de los datos generados por el simulador, permitiendo identificar patrones, tendencias y relaciones entre las variables del sistema.

4.8.1. Procesamiento de datos cuantitativos

Los datos cuantitativos se obtienen de las ejecuciones del simulador, donde se registran variables como número de iteraciones, estados promedio de las celdas, métricas de error y estabilidad de los patrones emergentes durante un periodo determinado. Posteriormente, estos datos se organizan, procesan y analizan mediante herramientas estadísticas y de visualización.

- **Análisis estadístico descriptivo:** Se calculan medidas de tendencia central y de dispersión con el fin de describir cuantitativamente el comportamiento de las variables analizadas y detectar posibles irregularidades o tendencias [**?**].
- **Visualización de datos:** Se elaboran gráficos de barras, histogramas y diagramas de dispersión para representar visualmente los resultados obtenidos, facilitando la comprensión de la dinámica del sistema y la comparación entre diferentes configuraciones experimentales.

El procesamiento y análisis de los datos se lleva a cabo utilizando herramientas computacionales. Los resultados obtenidos servirán como base para la interpretación y discusión en las etapas posteriores de la investigación.

4.9. Contextualización de la tecnología

El desarrollo de un simulador de sistemas complejos mediante autómatas celulares neuronales implica la integración de múltiples tecnologías, seleccionadas estratégicamente para satisfacer los requerimientos funcionales y técnicos del proyecto. Entre estos se incluye el diseño de una interfaz grafica de usuario intuitiva, la simulación de CA bidimensionales con diferentes estados, vecindades y funciones de transición, la personalización de condiciones iniciales y el entrenamiento de sistemas para la replicación de comportamientos brindados por el usuario.

La elección de estas tecnologías responde a criterios de compatibilidad, escalabilidad, facilidad de implementación y disposición de herramientas especificas para el procesamiento de datos e inteligencia artificial.

Con el fin de proporcionar una visión clara y compresiva de las bases técnicas del proyecto, en este capitulo se presentan tanto el área de estudio , elementos clave para orientar el desarrollo del simulador y validarlo, así como los fundamentos y el contexto de las tecnologías utilizadas, así como los entornos de trabajo que hacen posible este trabajo.

4.9.1. Fundamentos y contexto de las tecnologías

Un lenguaje de programación es una herramienta que permite el desarrollo de software y programas para computadora. Son empleados para diseñar programas encargados de definir el comportamiento físico y lógico de una computadora [**?**].

Debido a que la elección del entorno y lenguaje de programación resulta crítica para el éxito de cualquier proyecto, es importante considerar las metas y alcances del proyecto, puesto que no todos los lenguajes son óptimos para resolver ciertos tipos de problemas [**?**].

Entre los factores clave para la elección de un lenguaje de programación se encuentra la escalabilidad y el rendimiento, el costo de mantenimiento, la compatibilidad y el sistema de herramientas, entre otras [**?**].

Es por esto que, como lenguaje de programación se decidió utilizar C++/CUDA, para el núcleo de simulación y cálculos pesados en paralelo; y Python, para la interfaz, visualización, control de simulación y diseño de redes neuronales.

A continuación se presentan las definiciones pertinentes de los lenguaje de programación, frameworks y librerías utilizadas para el desarrollo de la aplicación móvil.

4.9.1.1. C++

C++ es un lenguaje de programación compilado y multiparadigma desarrollado como una extensión de C [**?**]. Se considera un lenguaje de bajo nivel ya que permite manipular directamente la memoria, lo que lo convierte en un lenguaje muy poderoso. Es orientado a objetos, permite la creación de clases y objetos y es ampliamente utilizado para aplicaciones que requieren un alto rendimiento.

Es utilizado en motores de videojuegos, software de sistemas, aplicaciones en tiempo real, simulaciones científicas y calculo numérico [**?**]. para esta investigación, se utilizaran las siguiente bibliotecas de C++:

- **Eigen:** biblioteca para álgebra lineal y manipulación eficiente de matrices y vectores [**?**].
- **tiny-dnn:** librería para redes neuronales ligeras, permitiendo definir la función de transición de los autómatas celulares neuronales [**?**].
- **pybind11:** facilita la integración de código C++ con Python, permitiendo exponer funciones y clases para análisis y visualización [**?**].
- **HDF5 C++ API:** para almacenar y leer grandes volúmenes de datos de simulación de manera eficiente [**?**].

4.9.1.2. CUDA

CUDA es una plataforma de computación paralela y un modelo de programación desarrollado por NVIDIA que permite utilizar la GPU para cálculos de propósito general. Se emplea para acelerar operaciones matemáticas y simulaciones que requieren gran cantidad de procesamiento paralelo, como el entrenamiento de redes neuronales [**?**].

4.9.1.3. Python

De acuerdo con la página oficial de la documentación de Pyhton [**?**], es un lenguaje de programación potente y fácil de aprender. Tiene estructuras de datos de alto nivel, sintaxis simple y elegante, un tipado dinámico, naturaleza interpretada, etc.

Además de ser gratuito, se puede integrar bien en todos los equipos y sistemas. En adición a esto, [**?**], señala que en los beneficios que Python incluye se encuentran los siguientes:

- Fácil lectura y comprensión de los programas.
- Mayor productividad al escribir un programa con menos líneas de código que en comparación con muchos otros lenguajes de programación.
- Se puede usar fácilmente con otros lenguajes de programación.
- Soporte rápido de la comunidad.
- Se puede trasladar a través de diferentes sistemas operativos de computadora.

En adición a lo anterior, cuenta con una amplia colección de librerías y frameworks diseñados para tareas que resultaron muy convenientes en el desarrollo de la aplicación móvil. Se detallan tanto librerías como frameworks a continuación.

- **Os:** es una librería que provee funcionalidades dependientes del sistema operativo como abrir un archivo, manipular rutas, leer archivos, crear archivos temporales, etc. [**?**].
- **NumPY** es una librería de Python especializada en el cálculo numérico y el análisis de datos. Es ideal para el manejo de grandes volúmenes de datos [**?**].

  Puede hacer operaciones hasta 50 veces mas rápido que las listas normales usando una objeto llamado arreglo que permite representaciones colecciones de datos en varias dimensiones [**?**].
- **CuPy** es una biblioteca de matrices de código abierto para la computación acelerada por GPU con Python. Utiliza bibliotecas del CUDA Toolkit para hacer un uso completo de la arquitectura GPU. Es compatible con NumPy y permite definir núcleos CUDA personalizados [**?**].

- **PyTorch** es una biblioteca de tensores optimizada para el aprendizaje profundo utilizando GPU y CPU [**?**]. Se emplea para crear redes neuronales, combinando la biblioteca de ML de Torch con una API de alto nivel basada en Python.

  Es uno de los marcos de ML destinados a comunidades académicas y de investigación por su flexibilidad y fácil uso, además de admitir una amplia variedad de estructuras de redes neuronales [**?**].

- **PyQt5 / PySide6** son herramientas que permiten usar Python como un lenguaje alternativo de desarrollo de aplicaciones a C++ en todas las plataformas soportadas [**?**]. PyQt5 también puede integrarse en aplicaciones basadas en C++ para permitir a los usuarios de dichas aplicaciones configurar o mejorar la funcionalidad de las mismas, mientras que PySide6 ofrece compatibilidad oficial con las bibliotecas Qt desarrolladas por The Qt Company [**?**].

- **h5py** es una librería que proporciona una interfaz entre Python y el formato de almacenamiento de datos jerárquico HDF5 (Hierarchical Data Format). Este formato permite manejar grandes volúmenes de datos de manera eficiente, organizándolos en estructuras tipo árbol con grupos y conjuntos de datos. Con h5py es posible almacenar, leer y modificar datos numéricos de gran tamaño directamente desde Python, lo cual resulta especialmente útil en simulaciones científicas que generan grandes matrices multidimensionales [**?**].

- **pybind11** es una biblioteca de C++ diseñada para exponer tipos y funciones de C++ en Python y viceversa, facilitando la creación de extensiones de Python basadas en código C++ [**?**]. Puede generar enlaces entre ambos lenguajes de manera compacta, eficiente y con un código más simple, posibilitando así integrar cálculos de alto rendimiento escritos en C++ dentro de entornos Python.

- **Pandas** es una librería especializada en el manejo y análisis de estructuras de datos. Entre sus principales características, [**?**] menciona:

  - Define nuevas estructuras de datos basadas en los arrays de la librería NumPy.
  - Permite leer y escribir fácilmente ficheros en formato CSV, Excel y bases de datos SQL.
  - Permite acceder a los datos mediante índices o nombres para filas y columnas.

  - Ofrece métodos para reordenar, dividir y combinar conjuntos de datos.

- **Matplotlib** De acuerdo con [**?**], Matplotlib es una librería de Pyhton especializada en la creación de gráficos en dos dimensiones.

  Además, mencionan que permite crear y personalizar los gráficos más comunes, entre los que se encuentran:

  - Diagramas de barras.
  - Histograma.
  - Diagramas de sectores.
  - Diagramas de caja y bigotes.

- **Scikit-learn** es una librería que cuenta con algoritmos de clasificación, regresión, clustering, y reducción de dimensionalidad, etc. Es compatible con otras librerías de Python [**?**].

  Entre sus principales características [**?**], destaca las siguientes:

  - Clustering.
  - Ensemble methods.
  - Validación cruzada.
  - Varios conjuntos de datos o datasets de prueba.
  - Reducción de la dimensionalidad.
  - Optimización o ajuste de hiperparámetros.
  - Feature selection.

4.9.2. Entornos de trabajo

El desarrollo del presente proyecto requiere de un entorno colaborativo, flexible y bien estructurado que facilite tanto la construcción como la validación de los distintos componentes del sistema. Para ello, se han seleccionado diversas plataformas que permiten organizar tareas, escribir y probar código, almacenar datos, así como documentar y presentar avances de manera clara y profesional.

4.9.2.1. Plataformas de desarrollo

Con el fin de cubrir las distintas etapas del proyecto, desde la implementación hasta la comunicación de resultados, se utilizaron múltiples herramientas que favorecen la eficiencia y el trabajo en equipo. Las siguientes plataformas han sido clave para mantener un flujo de trabajo continuo y bien coordinado.

- **Visual Studio Code** Comúnmente conocido como VSCode, es un editor de código gratuito, de código abierto y compatible con múltiples sistemas operativos. Fue creado por Microsoft, empresa con amplia trayectoria en el desarrollo de entornos integrados (IDE), lo que se refleja en la eficiencia y funcionalidad que ofrece esta herramienta, muy valorada por la comunidad de desarrolladores [**?**].

  Aunque su nombre puede sugerir una relación directa con Visual Studio, en realidad VSCode es una aplicación independiente, construida desde cero con tecnologías diferentes. Mientras Visual Studio se orienta principalmente a tecnologías propias de Microsoft, VSCode destaca por su flexibilidad y capacidad de adaptarse a prácticamente cualquier lenguaje de programación [**?**].

- **Github** Es una plataforma basada en la nube donde puedes almacenar, compartir, trabajar y versionar junto con otros usuarios para escribir código [**?**]. En adición a lo anterior, también permite realizar lo siguiente:

  - Presentar o compartir el trabajo.
  - Seguir y administrar los cambios en el código a lo largo del tiempo.
  - Dejar que otros usuarios revisen el código y realicen sugerencias para mejorarlo.
  - Colaborar en un proyecto compartido, sin preocuparse de que los cambios afectarán al trabajo de los colaboradores antes de que esté listo para integrarlos.

  El versionamiento, una de las características fundamentales de GitHub, es posible gracias al software de código abierto Git, en el que se basa GitHub [**?**].

- **Canva** Lanzada en 2013, Canva es una herramienta de diseño y publicación online que tiene como misión poner el poder del diseño al alcance de todo el mundo, para que cualquier persona pueda diseñar lo que quiera y publicar donde quiera [**?**].

- **TexStudio** TeXstudio es un entorno de escritura integrado para la creación de documentos LaTeX. Cuenta con numerosas características como resaltado de sintaxis, visor integrado, comprobación de referencias y varios asistentes. TeXstudio es de código abierto y está disponible para todos los principales sistemas operativos [**?**].

Habiendo definido el enfoque metodológico, justificado las decisiones técnicas del estudio y contextualizado la tecnología utilizada, se cuenta con una base sólida para la implementación y ejecución de los experimentos. Esto permite garantizar que las simulaciones se realicen bajo condiciones controladas, que los datos recolectados sean confiables y que los análisis posteriores aporten resultados válidos y pertinentes para responder a las preguntas de investigación planteadas.

Capítulo 5

Análisis de software

Hasta el momento conocemos la metodología utilizada para la investigación completa, sin embargo, es necesario especificar el desarrollo del software ya que es necesario garantizar el correcto diseño e implementación del simulador para poder realizar las pruebas adecuadas con el menor numero de errores. Es por esto que en este capitulo se detalla el proceso de desarrollo del simulador, lo que incluye la especificación de la metodología utilizada, requerimientos del sistema, y modelado, diseño e implementación de cada una de las entregas.

5.1. Metodología de desarrollo

Para el desarrollo se software se utiliza un modelo de entrega por etapas. Este modelo, como se observa en la figura 5.1, realiza las primeras etapas de forma secuencial y permite diseñar e implementar los componentes del proyecto por separado. Seguir este modelo permite entregar partes del proyecto y tener visibilidad del progreso.

Estas son algunas de las razones por la cual se opta por este modelo:

- El tamaño del equipo de desarrollo es reducido: Al tener un numero de integrantes reducido, resulta inviable desarrollar múltiples componentes de forma paralela.
- No existe un cliente: El desarrollo de este producto de software es unicamente un medio para el proceso experimental.

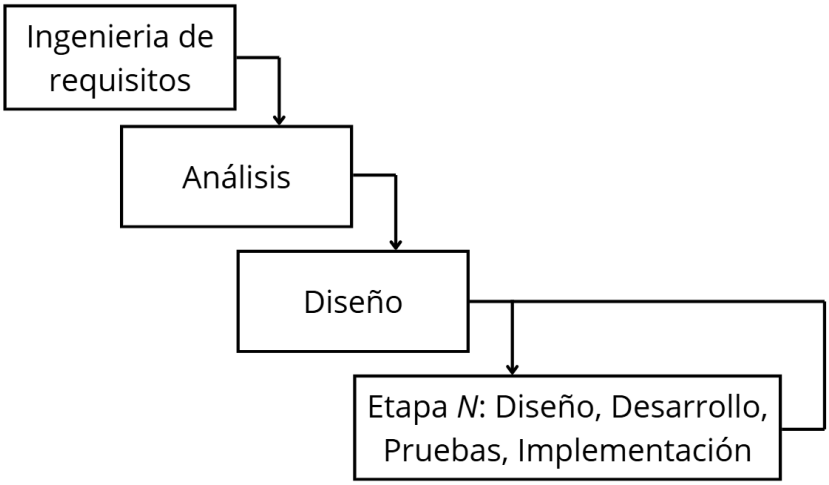


Figura 5.1: Representación grafica del modelo de entrega por etapas para el desarrollo del software

- No es un producto comercial: No se planea monetizar dado que no existe un cliente. Por lo que en la ingeniería de requisitos se consideran los necesarios propuestos por el equipo de investigación para satisfacer las necesidades tecnológicas.
- No hay procesos iterativos: Al no tener cliente no es necesario tener una validación externa, ni cambios significativos en los requerimientos del software.

5.2. Reglas de negocio

RN1 El estado de una celda en el tiempo $t + 1$ depende unicamente de su estado actual y de los estados de sus vecinos.

RN2 Para iniciar la simulación es necesario que el conjunto de estados, la topología de la vecindad, la condición de frontera y la función de transición hayan sido definidas.

RN3 Cuando la simulación esta en curso no se puede cambiar el conjunto de estados, la topología de la vecindad ni la función de transición.

RN4 El usuario puede manipular el estado de una célula y la condición de frontera en tiempo de ejecución.

RN5 Los datos importados deben de cumplir con el formato definido por el sistema para garantizar su correcta interpretación.

RN6 Los datos exportados deben conservar el mismo formato que los datos importados.

RN7 El usuario puede pausar, detener, reanudar o retroceder la simulación sin perdida de información.

RN8 Solo se permite una simulación activa por sesión.

5.3. Requerimientos

5.3.1. Requerimientos funcionales

Los requerimientos funcionales describen las capacidades que el sistema debe de ofrecer al usuario para cumplir con los objetivos establecidos.

RF1. El sistema debe mostrar una cuadrícula regular y finita que represente el espacio celular.

RF2. El sistema debe permitir al usuario interactuar con el espacio celular, asignando o modificando los estados de las celdas.

RF3. El sistema debe permitir al usuario definir el conjunto de estados, la topología de la vecindad local y las condiciones de frontera del espacio celular.

RF4. El sistema debe permitir la evolución del espacio celular mediante un único paso o de forma continua, ajustando la velocidad de actualización según las capacidades computacionales del sistema.

RF5. El sistema debe permitir la importación y exportación de configuraciones y evoluciones del espacio celular.

RF6. El sistema debe permitir al usuario establecer la función de transición de manera manual.

RF7. El sistema debe permitir configurar la red neuronal para imitar el comportamiento de una evolución señalada.

RF8. El sistema debe entrenar una red neuronal dados los parámetros del usuario y la evolución que desea replicar.

5.3.2. Requerimientos no funcionales

Los requerimientos no funcionales especifican las condiciones de calidad, desempeño y compatibilidad bajo las cuales el sistema debe de operar.

RNF1. El sistema debe poder ejecutarse en Fedora 42 sin requerir modificaciones en el código fuente.

RNF2. El sistema debe ser capaz de procesar al menos 10,000 celdas o registros por segundo utilizando una GPU NVIDIA RTX 5080.

RNF3. El sistema debe garantizar disponibilidad continua durante la ejecución, sin fallos o interrupciones críticas.

RNF4. La interfaz del sistema debe tener mensajes claros y funcionalidades bien definidas.

5.3.3. Requerimientos de sistema

5.3.3.1. Recomendado

Los siguientes requerimientos describen el entorno principal y recomendado de ejecución, en el cual el sistema garantiza un rendimiento óptimo y estabilidad durante los procesos de simulación:

- **Procesador:** Intel Core i9-14900K × 32 núcleos.
- **GPU:** NVIDIA GeForce RTX 5080.
- **Sistema operativo:** Fedora Linux 42 (Workstation Edition).
- **Entorno de escritorio:** GNOME 48.
- **Sistema de ventaneo:** Wayland.

5.3.3.2. Mínimo

De igual forma, se definen los requerimientos mínimos necesarios para la ejecución del simulador, considerando que el rendimiento y la velocidad de procesamiento estarán limitados por las capacidades del hardware disponible y el entrenamiento puede ser inviable:

- **Procesador:** Intel Core i7-1185G7 × 4 núcleos.
- **GPU:** Intel Corporation Raptor Lake-S GT1 [UHD Graphics 770].
- **Sistema operativo:** Fedora Linux 42 (Workstation Edition).
- **Entorno de escritorio:** GNOME 48.
- **Sistema de ventaneo:** Wayland.

5.4. Análisis de riesgos

El análisis de riesgos se realiza tomando en cuenta el tiempo estimado de desarrollo de aproximadamente de 6 meses, los recursos disponibles para el desarrollo tecnológico y las habilidades técnicas para el diseño de simuladores de autómatas celulares, computo paralelo, redes neuronales y programación orientada a eventos.

5.4.1. Identificación de riesgos

5.4.1.1. Riesgos técnicos

RT1 Incompatibilidad entre diferentes lenguajes de programación: los modulos desarrollados en Python, C++ y CUDA generan errores de integración al intercambiar información y sincronizar procesos.

RT2 Indisponibilidad de hardware: La tarjeta grafica no tiene el rendimiento suficiente.

RT3 Incompatibilidad entre tecnologías: El sistema operativo, los lenguajes de programación y los elementos de hardware no son compatibles para el desarrollo y ejecución.

RT4 Dependencias de librerías: La instalación y configuración de diferentes librerías varia según el sistema operativo.

RT5 Mala visualización: El renderizado de los componentes gráficos es incorrecto o esta mal optimizado.

5.4.1.2. Riesgos operativos

RO1 Documentación deficiente: La documentación no permite mantener ni ampliar el proyecto a futuro.

RO2 Problemas personales: Falta de motivación o problemas de salud física o mental se presentan durante el desarrollo.

RO3 Curva de aprendizaje elevada: Se encuentra una curva de aprendizaje elevada y se carece de experiencias con la tecnología a utilizar.

5.4.1.3. Riesgos de requerimientos

RR1 Requerimientos ambiguos: La definición incompleta o ambigua de los requerimientos puede provocar cambios en etapas avanzadas del desarrollo.

5.4.1.4. Riesgos de estimación

RE1 Subestimación del tiempo de desarrollo: La complejidad de enlazar diferentes tecnologías puede extender los plazos planeados.

5.4.1.5. Riesgos de alcance

RA1 Limitación del alcance funcional: Las restricciones de tiempo pueden dar pie a limitar el numero de funcionalidades requeridas.

RA2 Carga de trabajo imprevista: El poder de computo resulta insuficiente por lo que se reduce la complejidad de las simulaciones.

5.4.2. Probabilidad e impacto

Siguiendo la tabla 5.1, a continuación se enlistan los niveles de riesgo para cada riesgo identificado.

		Probabilidad				
		Nada probable	Levemente probable	Poco probable	Probable	Muy probable
Severidad	Nula	Bajo	Bajo	Bajo	Medio	Medio
	Menor	Bajo	Bajo	Medio	Medio	Medio
	Moderado	Medio	Medio	Medio	Alto	Alto
	Crítico	Medio	Medio	Alto	Alto	Muy alto
	Fatal	Medio	Alto	Alto	Muy alto	Muy alto

Tabla 5.1: Matriz de riesgos [35]

Riesgo tecnico	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Incompatibilidad entre diferentes lenguajes de programación	Poco probable	Crítico	Alto
Indisponibilidad de hardware	Levemente probable	Moderado	Medio
Incompatibilidad entre tecnologías	Probable	Fatal	Muy alto
Dependencias de librerías	Levemente probable	Menor	Bajo
Mala visualización	Poco probable	Menor	Medio

Tabla 5.2: Nivel de riesgos técnicos

Riesgo operativo	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Documentación deficiente	Probable	Moderado	Alto
Problemas personales	Poco probable	Crítico	Alto
Curva de aprendizaje elevada	Muy probable	Moderado	Alto

Tabla 5.3: Nivel de riesgos operativos

Riesgo de requerimientos	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Requerimientos ambiguos	Levemente probable	Menor	Bajo

Tabla 5.4: Nivel de riesgos de requerimientos

Riesgo de estimación	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Subestimación del tiempo de desarrollo	Probable	Fatal	Muy alto

Tabla 5.5: Nivel de riesgos de estimación

Riesgo de alcance	**Probabilidad**	**Impacto**	**Nivel de riesgo**
Limitación del alcance funcional	Poco probable	Moderado	Medio
Carga de trabajo imprevista	Probable	Crítico	Alto

Tabla 5.6: Nivel de riesgos de alcance

5.4.3. Plan de mitigación

Riesgos técnicos	
Incompatibilidad entre diferentes lenguajes	Verificar la documentación de compatibilidad entre los lenguajes de programación y realizar pruebas de transferencia de datos y sincronización previas al desarrollo.
Indisponibilidad de hardware	Asegurar la disponibilidad de equipos de respaldo o entornos virtuales alternativos para continuar el desarrollo en caso de fallos de hardware.
Incompatibilidad entre tecnologías	Realizar pruebas de compatibilidad del hardware y software en diferentes sistemas operativos y versiones antes de la integración.
Dependencias de librerías	Documentar y verificar las versiones de librerías utilizadas, además de considerar alternativas equivalentes que garanticen continuidad en caso de incompatibilidad.
Mala visualización	Realizar pruebas de usabilidad y ajustar la interfaz de acuerdo con las resoluciones de pantalla y dispositivos objetivo.

Tabla 5.7: Plan de mitigación de riesgos técnicos

Riesgos operativos	
Documentación deficiente	Establecer un estándar de documentación y realizar revisiones periódicas del avance para garantizar la calidad y completitud de los registros técnicos.
Problemas personales	Definir roles de respaldo o distribuir responsabilidades para minimizar el impacto de ausencias o contratiempos personales.
Curva de aprendizaje elevada	Programar capacitaciones iniciales y establecer un periodo de adaptación con apoyo técnico y materiales de referencia.

Tabla 5.8: Plan de mitigación de riesgos operativos

Riesgos de requerimientos	
Requerimientos ambiguos	Realizar reuniones periódicas con los usuarios o stakeholders para aclarar requerimientos y validar el entendimiento del equipo de desarrollo.

Tabla 5.9: Plan de mitigación de riesgos de requerimientos

Riesgos de estimación	
Subestimación del tiempo de desarrollo	Dividir el proyecto en fases pequeñas y manejables, aplicando seguimiento semanal y reajuste de cronograma según los resultados obtenidos.

Tabla 5.10: Plan de mitigación de riesgos de estimación

Riesgos de alcance	
Limitación del alcance funcional	Definir claramente los objetivos y alcances desde el inicio, con aprobación formal de los responsables del proyecto antes de iniciar nuevas funcionalidades.
Carga de trabajo imprevista	Implementar revisiones periódicas del plan de trabajo y ajustar el alcance en función de los recursos disponibles y los resultados obtenidos.

Tabla 5.11: Plan de mitigación de riesgos de alcance

5.4.4. Plan de contingencia

A continuación se describen los planes de contingencia para los riesgos de nivel fatal:

RT3 Incompatibilidad entre tecnologías.

RE1 Subestimación del tiempo de desarrollo.

Nombre del riesgo	**Incompatibilidad entre tecnologías**
Probabilidad	Probable
Impacto	Fatal
Plan de evasión	Realizar pruebas de integración tempranas entre las distintas tecnologías implicadas. Documentar las versiones, dependencias y configuraciones utilizadas para garantizar compatibilidad. Establecer un entorno de desarrollo unificado que minimice diferencias entre plataformas.
Mitigación	Sustituir o actualizar los componentes que generen incompatibilidad. Implementar adaptadores o módulos intermedios que permitan la interoperabilidad entre sistemas. Reasignar recursos técnicos para resolver los conflictos de integración y evitar retrasos prolongados.

Tabla 5.12: Plan de contingencia para incompatibilidad entre tecnologías

Nombre del riesgo	**Subestimación del tiempo de desarrollo**
Probabilidad	Probable
Impacto	Fatal
Plan de evasión	Realizar estimaciones realistas basadas en experiencias previas y tareas medibles. Incluir márgenes de tiempo de reserva en el cronograma para absorber imprevistos. Efectuar reuniones de seguimiento semanales para detectar desviaciones tempranas.
Mitigación	Reasignar recursos o personal adicional para recuperar el tiempo perdido. Reducir temporalmente el alcance funcional o posponer características no críticas. Reajustar el cronograma de entregas y comunicar los cambios al equipo y partes interesadas.

Tabla 5.13: Plan de contingencia para subestimación del tiempo de desarrollo

5.5. Estudio de factibilidad

El proyecto cuenta con los recursos de hardware necesarios para llevar a cabo simulaciones de alto nivel computacional. El equipo de cómputo disponible permite implementar técnicas de paralelización, asegurando la capacidad de realizar múltiples inferencias en tiempos razonables. Adicionalmente, se dispone de herramientas tecnológicas de alto rendimiento que brindan la ventaja computacional requerida para ejecutar la simulación de manera eficiente.

En cuanto a la factibilidad económica, la mayor parte del software utilizado es de libre acceso, incluyendo Python, C++, Visual Studio Code y TeXstudio. Se utiliza también Wolfram Mathematica, que requiere una licencia anual de 195 dólares, la cual se encuentra disponible de manera gratuita para estudiantes del IPN, garantizando así la viabilidad financiera del proyecto.

Desde el punto de vista operativo, el diseño de la interfaz facilita la definición de reglas y la ejecución de simulaciones, optimizando el flujo de trabajo del grupo de investigadores. Esto permite acelerar significativamente el proceso de experimentación y análisis de resultados, maximizando la eficiencia del equipo durante el desarrollo del proyecto.

Finalmente, se cumple la factibilidad temporal, ya que el cronograma de desarrollo estimado permite realizar todas las simulaciones necesarias para el análisis posterior, asegurando que el proyecto se pueda completar dentro del tiempo previsto.

5.6. Casos de uso

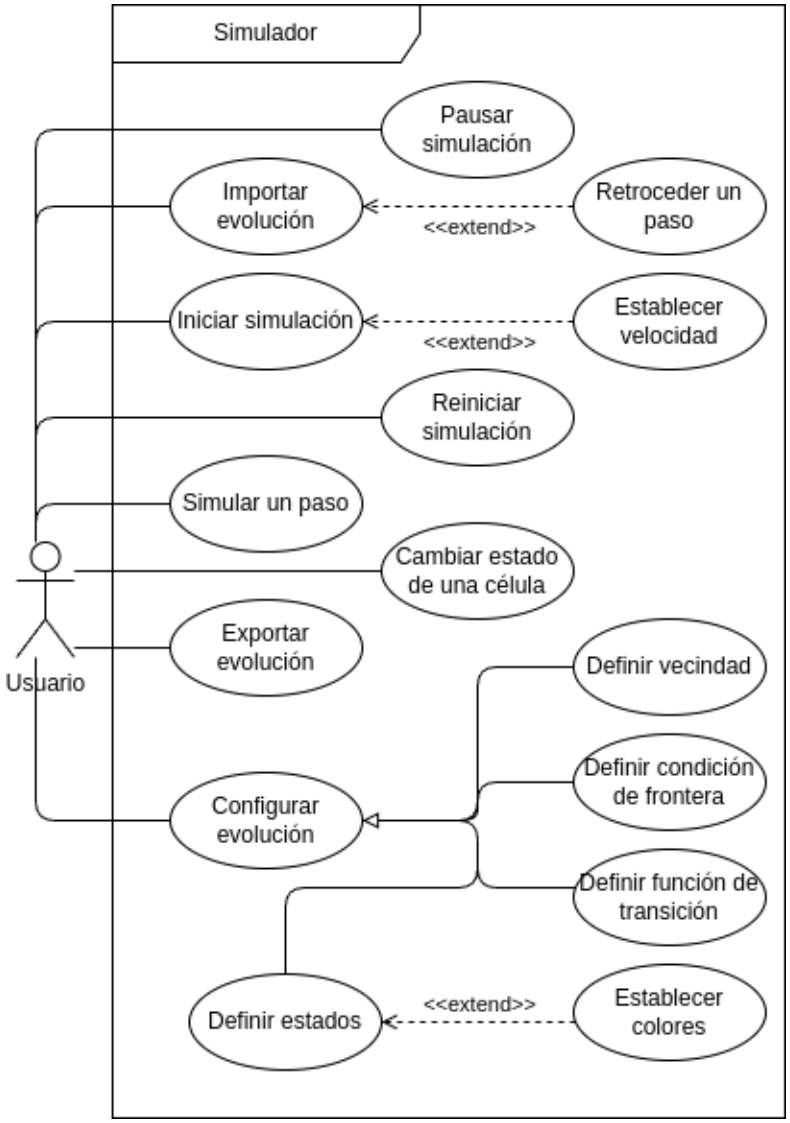


Figura 5.2: Diagrama de casos de uso

5.7. Diccionario

5.7.1. Entradas

- Definición de la configuración del CA.

    - BTN1 Vecindad
    - BTN2 Configuración de la vecindades
    - BTN3 Guardar vecindad
    - BTN4 Cancelar cambio de vecindad
    - BTN5 Condición de frontera
    - BTN6 Cambiar estados
    - BTN7 Rechazar estados
    - BTN8 Aceptar estados
    - BTN9 Cancelar estados
    - BTN10 Aplicar regla
    - BTN11 Color
    - BTN12 Célula
- Simulación.
    - BTN13 Simular
    - BTN14 Pausa
    - BTN15 Acelerar
    - BTN16 Frenar
    - BTN17 Paso
    - BTN18 Retroceder
    - BTN19 Reiniciar
- Importación y exportación.
    - BTN20 Importar
    - BTN21 Exportar
- Campos de texto.
    - IN1 Tamaño de la vecindad
    - IN2 Regla
    - IN3 Estados

5.7.2. Mensajes

- MSG1 Vecindad generada
- MSG2 Vecindad generada con incompatibilidad
- MSG3 Incompatibilidad de estados
- MSG4 Regla generada
- MSG5 Importación requerida
- MSG6 Limite de evolución alcanzado
- MSG7 Importación exitosa
- MSG8 Formato invalido

5.7.3. Interfaces graficas

- UI1 Simulación: Elemento que contiene la cuadricula del espacio celular y muestra la evolución del CA donde cada célula actúa como un botón BTN12 Célula.
- UI2 Control: Elemento que contiene el control sobre la simulación del espacio celular.
- UI3 Vecindad: Ventana emergente que contiene un espacio celular donde cada célula actúa como un botón BTN12 Célula. Muestra la célula central y permite al usuario seleccionar aquellas células que quiere que formen parte de la vecindad.
- UI4 Configuración: Elemento que permite realizar la configuración de la condición de frontera, los estados y la función de transición.

5.8. Especificaciones de casos de uso

5.8.1. Definir vecindad

Actor	Usuario
Descripción	Se define el conjunto de posiciones relativas de cada célula que forman parte de la vecindad.
Tipo	Primario
Entradas	▪ Desde mouse BTN1 Vecindad ▪ Desde mouse BTN12 Célula ▪ Desde mouse BTN3 Guardar vecindad ▪ Desde mouse BTN4 Cancelar cambio de vecindad
Salidas	▪ Mensaje MSG1 Vecindad generada. ▪ Mensaje MSG2 Vecindad generada con incompatibilidad
Precondición	Ninguna
Postcondición	Ninguna
Errores	▪ Simulando: Se intenta cambiar la vecindad mientras esta una simulación en proceso. ▪ Cancelar: Se cancela el cambio de vecindad. ▪ Incompatibilidad: La regla es incompatible con la nueva vecindad seleccionada.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN1 Vecindad en UI4 Configuración. 3. Selecciona los elementos que forman parte de la vecindad presionando los botones BTN12 Célula. 4. Presiona el botón BTN3 Guardar vecindad.	2. Abre la interfaz UI3 Vecindad. 5. Actualiza la vecindad. 6. Muestra el mensaje MSG1 Vecindad generada.

Flujo alterno 1: Simulando	
Actor	**Sistema**
1.1. Presiona el botón BTN1 Vecindad en UI4 Configuración mientras la simulación esta activa. 4.1. Selecciona los elementos que forman parte de la vecindad presionando los botones BTN12 Célula. 5.1. Presiona el botón BTN3 Guardar vecindad.	2.1. Detiene la simulación. 3.1. Abre la interfaz UI3 Vecindad. 6.1. Actualiza la vecindad. 7.1. Muestra el mensaje MSG1 Vecindad generada.

Flujo alterno 2: Cancelar	
Actor	**Sistema**
4.2. Presiona el botón de BTN4 Cancelar cambio de vecindad.	5.2. No se actualiza la vecindad.

Flujo alterno 3: Incompatibilidad	
Actor	**Sistema**
3.3. Selecciona los elementos que forman parte de la vecindad presionando los botones BTN12 Célula pero el nuevo tamaño es diferentes al anterior. 4.3. Presiona el botón BTN3 Guardar vecindad.	5.3. Actualiza la vecindad. 6.3. Muestra el mensaje MSG2 Vecindad generada con incompatibilidad. 7.3. Muestra el mensaje MSG1 Vecindad generada.

5.8.2. Definir condición de frontera

Actor	Usuario
Descripción	Se selecciona entre diferentes opciones de condiciones de frontera.
Tipo	Primario
Entradas	▪ Desde mouse BTN5 Condición de frontera
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Ninguna
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN5 Condición de frontera desde UI4 Configuración.	2. Actualiza la condición de frontera a la siguiente configuración.

5.8.3. Definir estados

Actor	Usuario
Descripción	Se define el tamaño del conjunto de estados.
Tipo	Primario
Entradas	▪ Desde teclado IN3 Estados ▪ Desde mouse BTN6 Cambiar estados
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Ninguna
Errores	▪ Simulando: El usuario intenta cambiar los estados mientras esta en ejecución. ▪ Incompatibilidad: El numero de estados es menor y hay estados inexistentes en el espacio celular.

Flujo básico	
Actor	**Sistema**
1. Ingresa el numero de estados en el campo IN3 Estados desde UI4 Configuración. 2. Presiona el botón BTN6 Cambiar estados.	3. Cambia el conjunto de estados.

Flujo alterno 1: Simulando	
Actor	**Sistema**
2.1. Presiona el botón BTN6 Cambiar estados mientras la simulación esta en ejecución.	3.1. Detiene la simulación. 4.1. Cambia el conjunto de estados.

Flujo alterno 2: Incompatibilidad (Aceptar)	
Actor	**Sistema**
1.2. Ingresa el numero de estados en el campo IN3 Estados desde UI4 Configuración menor al actual. 2.2. Presiona el botón BTN6 Cambiar estados. 4.2. Presiona el botón BTN8 Aceptar estados.	3.2. Muestra el mensaje MSG3 Incompatibilidad de estados. 5.2. Cambia el conjunto de estados. 6.2. Establece a “0” todos los estados inexistentes.

Flujo alterno 2.1: Incompatibilidad (Rechazar)	
Actor	**Sistema**
1.2. Ingresa el numero de estados en el campo IN3 Estados desde UI4 Configuración menor al actual. 2.2. Presiona el botón BTN6 Cambiar estados. 4.2. Presiona el botón BTN7 Rechazar estados.	3.2. Muestra el mensaje MSG3 Incompatibilidad de estados. 5.2. No hace ningún cambio.

5.8.4. Definir función de transición

Actor	Usuario
Descripción	Se configura la regla que define el comportamiento del CA.
Tipo	Primario
Entradas	▪ Desde teclado IN2 Regla ▪ Desde mouse BTN10 Aplicar regla
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Ninguna
Errores	▪ Simulando: El cambio de regla se intenta aplicar cuando la simulación esta en ejecución. ▪ Fuera de rango: La regla establecida esta fuera del rango.

Flujo básico	
Actor	**Sistema**
1. Ingresa la regla en el campo IN2 Regla desde UI4 Configuración. 2. Presiona el botón BTN10 Aplicar regla.	3. Actualiza la regla. 4. Muestra el mensaje MSG4 Regla generada.
Flujo alterno 1: Simulando	
Actor	**Sistema**
2.1. Presiona el botón BTN10 Aplicar regla mientras la simulación esta activa.	3.1. Pausa la simulación. 4.1. Actualiza la regla. 5.1. Muestra el mensaje MSG4 Regla generada.

Flujo alterno 2: Fuera de rango	
Actor	**Sistema**
	3.3. La regla no es compatible, se ajusta a la configuración del CA. 4.3. Actualiza la regla. 5.3. Muestra el mensaje MSG4 Regla generada.

5.8.5. Establecer colores

Actor	Usuario
Descripción	Se define un color asociado a cada estado.
Tipo	Secundario
Entradas	▪ Desde mouse BTN11 Color
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Ninguna
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Selecciona un estado en UI4 Configuración. 2. Presiona el botón de BTN11 Color. 4. Selecciona un color.	3. Abre un panel de colores. 5. Asigna ese color a ese estado. 6. Cambia el color de todos las celular con en ese estado en el espacio celular.

5.8.6. Iniciar simulación

Actor	Usuario
Descripción	El sistema inicia la evolución en pasos de tiempo constante.
Tipo	Primario
Entradas	▪ Desde mouse BTN13 Simular
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Pausar simulación
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN13 Simular desde UI2 Control.	2. Inicia la simulación. Cada paso evolutivo se realiza a una velocidad constante. 1. Inhabilita el botón BTN13 Simular. 2. Habilita el botón BTN14 Pausa.

5.8.7. Pausar simulación

Actor	Usuario
Descripción	Se detiene la simulación.
Tipo	Primario
Entradas	▪ Desde mouse BTN14 Pausa.
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Iniciar simulación.
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN14 Pausa desde UI2 Control.	2. Detiene la simulación. 3. Inhabilita el botón BTN14 Pausa. 4. Habilita el botón BTN13 Simular.

5.8.8. Establecer velocidad

Actor	Usuario
Descripción	Se configura la velocidad de actualización del CA cuando se inicia la simulación.
Tipo	Secundario
Entradas	▪ Desde mouse BTN15 Acelerar ▪ Desde mouse BTN16 Frenar
Salidas	Ninguna
Precondición	Iniciar simulación
Postcondición	Ninguna
Errores	▪ Velocidad limite: Se intenta disminuir o aumentar la velocidad cuando ya esta en su velocidad mínima o máxima posible respectivamente.

Flujo básico (Acelerar)	
Actor	**Sistema**
1. Presiona BTN15 Acelerar desde UI2 Control.	2. Aumenta la velocidad de la simulación.

Flujo básico (Frenar)	
Actor	**Sistema**
1. Presiona BTN16 Frenar desde UI2 Control.	2. Disminuye la velocidad de la simulación.

Flujo alterno 1: Velocidad limite	
Actor	**Sistema**
1.1. Presiona BTN15 Acelerar o BTN16 Frenar desde UI2 Control cuando esta en la velocidad limite máxima o mínima respectivamente.	2.1. No realiza ningún cambio, mantiene la velocidad actual.

5.8.9. Simular un paso

Actor	Usuario
Descripción	Se realiza la simulación del CA pero unicamente en un paso.
Tipo	Primario
Entradas	▪ Desde mouse BTN17 Paso
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Ninguna
Errores	▪ Velocidad máxima: Se intenta dar un paso cuando la simulación esta activa en la velocidad máxima.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN17 Paso desde UI2 Control.	2. La simulación avanza un paso.

Flujo alterno 1: Velocidad máxima	
Actor	**Sistema**
	2.1. No realiza ningún cambio y mantiene la velocidad actual.

5.8.10. Retroceder un paso

Actor	Usuario
Descripción	Cuando se importa una evolución, se retrocede la simulación para visualizar evoluciones pasadas.
Tipo	Secundario
Entradas	▪ Desde mouse BTN18 Retroceder
Salidas	▪ Mensaje MSG5 Importación requerida
Precondición	Importar evolución
Postcondición	Ninguna
Errores	▪ Falta importación: El usuario intenta retroceder la simulación cuando no ha importado ningún archivo. ▪ Configuración inicial: EL usuario intenta retroceder la simulación de la configuración inicial.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN18 Retroceder desde UI2 Control.	2. La simulación retrocede un paso.

Flujo alterno 1: Falta importación	
Actor	**Sistema**
	2.1. No se ha importado la evolución. Bloquea el botón y muestra el mensaje MSG5 Importación requerida.

Flujo alterno 2: Configuración inicial	
Actor	**Sistema**
3.2. Presiona el botón BTN18 Retroceder desde UI2 Control estando en la configuración inicial del archivo (No hay mas configuraciones antes).	4.2. No realiza ningún cambio y muestra el mensaje MSG6 Limite de evolución alcanzado.

5.8.11. Reiniciar simulación

Actor	Usuario
Descripción	Se reinicia la simulación a la ultima cuadricula que fue modificada mientras la simulación estaba en pausa.
Tipo	Primario
Entradas	▪ Desde mouse BTN19 Reiniciar.
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Ninguna
Errores	Ninguno

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN19 Reiniciar desde UI2 Control.	2. Regresa la configuración de la cuadricula a la ultima versión modifica mientras estaba en pausa.

5.8.12. Cambiar estado de una célula

Actor	Usuario
Descripción	Se cambia manualmente la configuración del espacio celular, seleccionando el estado de cada célula.
Tipo	Primario
Entradas	▪ Desde mouse BTN12 Célula
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Ninguna
Errores	Ninguno

Flujo básico (Siguiente estado)	
Actor	**Sistema**
1. Presiona una célula BTN12 Célula desde UI1 Simulación.	2. La célula cambia al estado siguiente estado.

Flujo básico (Selección de estado)	
Actor	**Sistema**
1. Selecciona un estado desde UI4 Configuración. 2. Presiona una célula BTN12 Célula desde UI1 Simulación.	3. La célula cambia al estado presionado en UI4 Configuración.

5.8.13. Importar evolución

Actor	Usuario
Descripción	Importa un archivo que contiene el histórico de la evolución de un CA particular.
Tipo	Primario
Entradas	▪ Desde mouse BTN20 Importar
Salidas	▪ Mensaje MSG7 Importación exitosa ▪ Mensaje MSG8 Formato invalido
Precondición	Ninguna
Postcondición	Ninguna
Errores	▪ Formato invalido: El formato del archivo es incompatible o la información que contiene esta corrompida.

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN20 Importar desde UI4 Configuración. 3. Selecciona el archivo ".ca" que contiene la evolución.	2. Abre el explorador de archivos. 4. Carga la evolución del CA y muestra el mensaje MSG7 Importación exitosa.

Flujo alterno 1: Formato invalido	
Actor	**Sistema**
3.1. Selecciona un archivo invalido.	4.1. No realiza ningún cambio y muestra el mensaje MSG8 Formato invalido.

5.8.14. Exportar evolución

Actor	Usuario
Descripción	Se guarda el historial evolutivo obtenido desde el ultimo cambio realiza mientras la simulación estaba en pausa.
Tipo	Primario
Entradas	▪ Desde mouse BTN21 Exportar
Salidas	Ninguna
Precondición	Ninguna
Postcondición	Ninguna
Errores	▪ Cancelación

Flujo básico	
Actor	**Sistema**
1. Presiona el botón BTN21 Exportar desde UI4 Configuración. 4. Selecciona la ruta y el nombre del archivo.	2. Guarda la configuración y el historial evolutivo. 3. Abre el explorador de archivos. 5. Guarda el archivo creado en la ruta especificada.

Flujo alterno 1: Cancelación	
Actor	**Sistema**
4.1. Presiona el botón de cancelar.	5.1. No guarda el archivo y no realiza cambios.

Capítulo 6

Diseño de software

6.1. Arquitectura

La arquitectura del sistema se basa en el patrón de diseño **Modelo Vista Controlador (MVC)**, el cual resulta adecuado para aplicaciones que combinan lógica de negocio, manipulación de datos y una interfaz gráfica interactiva [36]. Este patrón favorece la separación de responsabilidades, la mantenibilidad y la extensibilidad del sistema.

- **Modelo:** Gestiona los datos y la lógica principal del autómata celular, incluyendo la definición de estados, reglas, vecindad y condiciones de frontera.
- **Vista:** Se encarga de la presentación visual de la información y la interacción con el usuario mediante la interfaz gráfica.
- **Controlador:** Actúa como intermediario entre el modelo y la vista, gestionando los eventos de usuario y actualizando el modelo o la interfaz según corresponda.

6.1.1. Vista

La vista está implementada en *Python* utilizando el framework **PyQt5**. Esta capa proporciona la interfaz gráfica principal del simulador, permitiendo visualizar el espacio celular, modificar parámetros y controlar la ejecución de la simulación (inicio, pausa, reinicio, etc.). Además, se incluyen componentes dinámicos que

permiten la selección de colores, la configuración de reglas y la representación visual del estado del sistema en tiempo real.

6.1.2. Modelo

El modelo encapsula la lógica del autómata celular. Esta capa está implementada en *C++* con el fin de optimizar el rendimiento en las operaciones de actualización de celdas y el procesamiento paralelo. El modelo gestiona la estructura del espacio celular, los estados, la topología de la vecindad, las condiciones de frontera y la regla de transición, garantizando un funcionamiento eficiente y escalable.

6.1.3. Controlador

El **controlador** coordina la comunicación entre la vista y el modelo. Implementado en Python, este componente recibe los eventos generados por la interfaz (por ejemplo, el cambio de número de estados o la selección de una regla), actualiza el modelo y refleja los resultados en la interfaz gráfica. Además, sincroniza los procesos de simulación y representación visual, permitiendo la ejecución paralela y una interacción fluida entre el usuario y la lógica del sistema.

6.2. Interfaces de usuarios

A continuación se muestran las interfaces desarrolladas en PyQt5. Se busca mantener un estructura intuitiva y modular que facilite la configuración, ejecución y visualización del CA.

- **Interfaz principal**

  La interfaz principal se divide en tres secciones: el espacio de evolución, ubicado en la parte superior izquierda, donde se visualiza la cuadrícula en la que evoluciona el autómata celular; el área de controles, situada en la parte inferior izquierda, que permite manipular la simulación (iniciar, pausar, reiniciar, avanzar o retroceder); y la sección de configuración, ubicada a la derecha, donde se especifican parámetros como la vecindad, las condiciones de frontera, los estados, los colores asociados y la regla de transición (Figura 6.1).

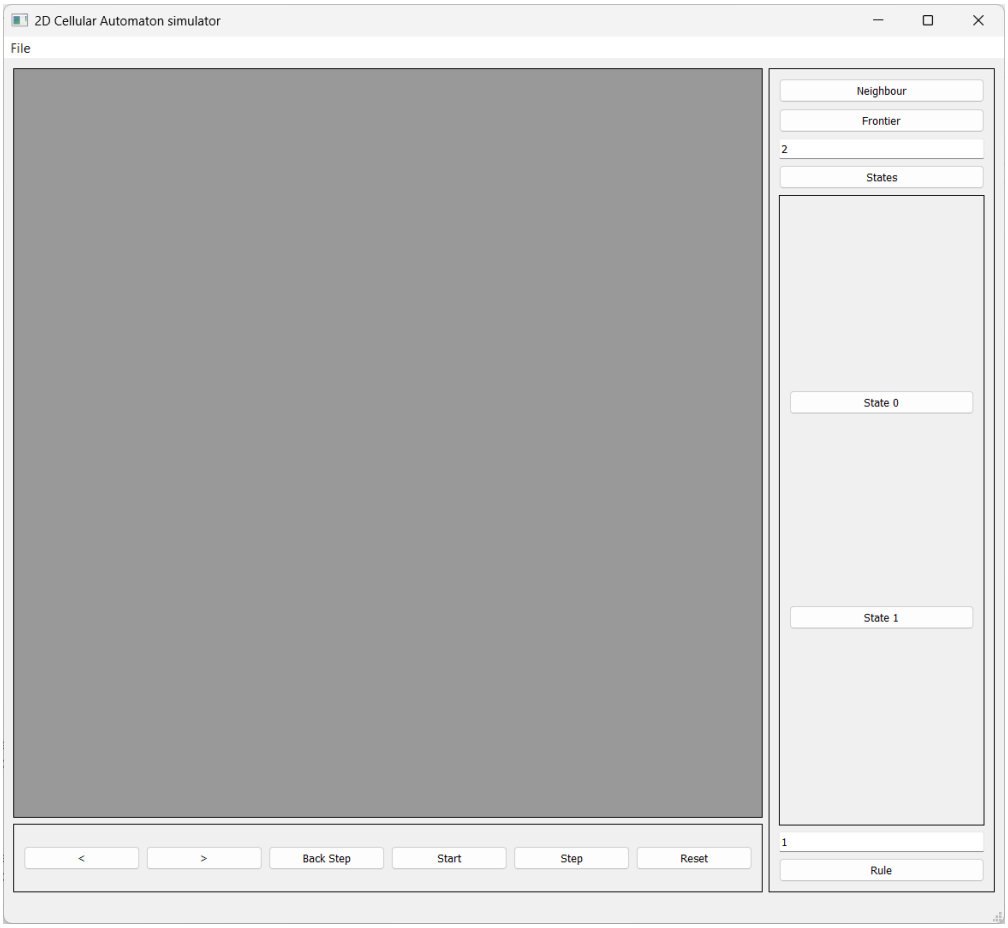


Figura 6.1: Interfaz de usuario principal

- **Interfaz de vecindad**

  Esta interfaz permite al usuario definir la topología de la vecindad que determinará la dinámica del CA. Para ello, se presenta un espacio celular de menor tamaño en el cual se pueden activar o desactivar las celdas que formarán parte de la vecindad (Figura 6.2).



Figura 6.2: Interfaz de usuario que define la vecindad

- **Interfaz para definir color**

  Esta ventana utiliza un componente nativo de **PyQt5** para la selección de colores. Permite al usuario asignar un color a cada estado del autómata (Figura 6.3).

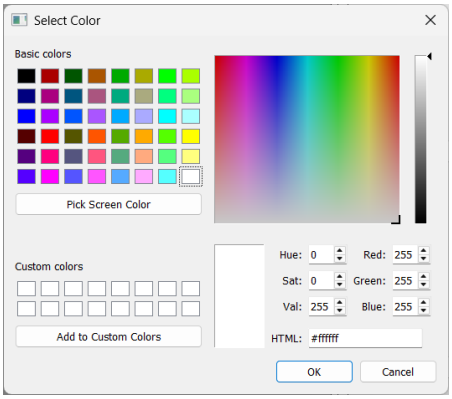


Figura 6.3: Interfaz de usuario para definir el color

Bibliografía

[1] J. Damerow. (2025) John von neumann's cellular automata. Accedido el 23 de julio de 2025. [Online]. Available: https://embryo.asu.edu/pages/john-von-neumanns-cellular-automata

[2] E. W. Weisstein, "Cellular automaton," https://mathworld.wolfram.com/CellularAutomaton.html, 2025, from MathWorld–A Wolfram Resource, accedido el 23 de julio de 2025.

[3] S. Wolfram, "Cellular automata as simple self-organizing systems," Caltech, Tech. Rep. CALT-68-938, 1982, general Theoretical Physics, 11 páginas.

[4] D. Grattarola, L. Livi, and C. Alippi, "Learning graph cellular automata," in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 20 983–20 994. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/af87f7cdcda223c41c3f3ef05a3aaeea-Paper.pdf

[5] P. Gács, "Probabilistic cellular automata with andrei toom," *Brazilian Journal of Probability and Statistics*, vol. 38, no. 2, Jun. 2024, accedido el 23 de julio de 2025. [Online]. Available: https://doi.org/10.1214/24-bjps605

[6] H. Hartman, P. Tamayo, and W. Klein, "Inhomogeneous cellular automata and statistical mechanics," *Complex Systems*, vol. 1, no. 2, 1987, massachusetts Institute of Technology and Boston University. [Online]. Available: https://content.wolfram.com/uploads/sites/13/2018/02/01-2-2.pdf

[7] L. Becker. (2025) Continuous cellular automata – hegl. HEGL – Heidelberg Experimental Geometry Lab, accedido el 23 de julio de 2025. [Online]. Available: https://hegl.mathi.uni-heidelberg.de/continuous-cellular-automata

[8] W. Gilpin, "Cellular automata as convolutional neural networks," *Physical Review E*, vol. 100, no. 3, Sep. 2019, accedido el 29 de julio de 2025. [Online]. Available: https://doi.org/10.1103/PhysRevE.100.032402

[9] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, 2020, https://distill.pub/2020/growing-ca.

[10] Fundación BBVA, "Diccionario - fundación bbva," https://www.fbbva.es/diccionario/retcula, 2025, accedido el 25 de julio de 2025.

[11] S. Wolfram, *A New Kind of Science*. Champaign, IL: Wolfram Media, 2002.

[12] *Asynchronicity in Neural Cellular Automata*, ser. ALIFE 2022: The 2022 Conference on Artificial Life, vol. ALIFE 2021: The 2021 Conference on Artificial Life, 07 2021. [Online]. Available: https://doi.org/10.1162/isal_a_00461

[13] A. Mordvintsev, E. Randazzo, and C. Fouts, "Growing isotropic neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2205.01681

[14] *Emergent Multiscale Structures and Generative Potential of Isotropic Neural Cellular Automata*, ser. ALIFE 2022: The 2022 Conference on Artificial Life, vol. ALIFE 2024: Proceedings of the 2024 Artificial Life Conference, 07 2024. [Online]. Available: https://doi.org/10.1162/isal_a_00745

[15] E. Randazzo, A. Mordvintsev, and C. Fouts, "Growing steerable neural cellular automata," 2023. [Online]. Available: https://arxiv.org/abs/2302.10197

[16] E. Randazzo, A. Mordvintsev, E. Niklasson, M. Levin, and S. Greydanus, "Self-classifying mnist digits," *Distill*, 2020, https://distill.pub/2020/selforg/mnist.

[17] E. Randazzo, A. Mordvintsev, E. Niklasson, and M. Levin, "Adversarial reprogramming of neural cellular automata," *Distill*, 2021, https://distill.pub/selforg/2021/adversarial.

[18] D. Zhang, C. Choi, J. Kim, and Y. M. Kim, "Learning to generate 3d shapes with generative cellular automata," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=rABUmU3ulQh

[19] S. Sudhakaran, D. Grbic, S. Li, A. Katona, E. Najarro, C. Glanois, and S. Risi, "Growing 3d artefacts and functional machines with neural cellular automata," 2021. [Online]. Available: https://arxiv.org/abs/2103.08737

[20] K. Horibe, K. Walker, and S. Risi, "Regenerating soft robots through neural cellular automata," 2021. [Online]. Available: https://arxiv.org/abs/2102.02579

[21] E. Niklasson, A. Mordvintsev, E. Randazzo, and M. Levin, "Self-organising textures," *Distill*, 2021, https://distill.pub/selforg/2021/textures.

[22] E. R. Alexander Mordvintsev, Eyvind Niklasson, "Texture generation with neural cellular automata," *CoRR*, vol. abs/2105.07299, 2021. [Online]. Available: https://arxiv.org/abs/2105.07299

[23] A. Mordvintsev and E. Niklasson, "μnca: Texture generation with ultra-compact neural cellular automata," *CoRR*, vol. abs/2111.13545, 2021. [Online]. Available: https://arxiv.org/abs/2111.13545

[24] E. Pajouheshgar, Y. Xu, A. Mordvintsev, E. Niklasson, T. Zhang, and S. Süsstrunk, "Mesh neural cellular automata," 2024. [Online]. Available: https://arxiv.org/abs/2311.02820

[25] E. Pajouheshgar, Y. Xu, A. Abbasi, A. Mordvintsev, W. Jakob, and S. Süsstrunk, "Neural cellular automata: From cells to pixels," 2025. [Online]. Available: https://arxiv.org/abs/2506.22899

[26] A. H. Ruiz, A. Vilalta, and F. Moreno-Noguer, "Neural cellular automata manifold," 2021. [Online]. Available: https://arxiv.org/abs/2006.12155

[27] K. Frans, "Stampca: Growing emoji with conditional neural cellular automata," *kvfrans.com*, 2021. [Online]. Available: http://kvfrans.com/stampca-conditional-neural-cellular-automata/

[28] M. Chen and Z. Wang, "Image generation with neural cellular automatas," 2020. [Online]. Available: https://arxiv.org/abs/2010.04949

[29] R. B. Palm, M. González-Duque, S. Sudhakaran, and S. Risi, "Variational neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2201.12360

[30] M. Tesfaldet, D. Nowrouzezahrai, and C. Pal, "Attention-based neural cellular automata," 2022. [Online]. Available: https://arxiv.org/abs/2211.01233

[31] J. Tang, S. Kumar, L. De Lorenzis, and E. Hosseini, "Neural cellular automata for solidification microstructure modelling," *Computer Methods in Applied Mechanics and Engineering*, vol. 414, p. 116197, Sep. 2023. [Online]. Available: http://dx.doi.org/10.1016/j.cma.2023.116197

[32] G. Li, C. Leefmans, J. Williams *et al.*, "Deep learning with photonic neural cellular automata," *Light: Science & Applications*, vol. 13, no. 283, 2024, accedido el 29 de julio de 2025. [Online]. Available: https://doi.org/10.1038/s41377-024-01651-7

[33] C. Grasso and J. Bongard, "Empowered neural cellular automata," ser. GECCO '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 108–111. [Online]. Available: https://doi.org/10.1145/3520304.3529067

[34] A. D. Richardson, T. Antal, R. A. Blythe, and L. J. Schumacher, "Learning spatio-temporal patterns with neural cellular automata," 2024. [Online]. Available: https://arxiv.org/abs/2310.14809

[35] Pymes y Calidad 2.0, "¿cómo se hace una matriz de riesgo? - ventajas y desventajas," https://www.pymesycalidad20.com/como-se-hace-una-matriz-de-riesgo-ventajas-y-desventajas.html, oct 2025, accedido el 28 de octubre de 2025.

[36] MDN Web Docs, "Mvc - glosario de mdn web docs," https://developer.mozilla.org/es/docs/Glossary/MVC, oct 2025, accedido el 28 de octubre de 2025.